

Universidades de Burgos, León y
Valladolid

Máster universitario

Inteligencia de Negocio y Big Data en Entornos Seguros



**TFM del Máster Inteligencia de Negocio
y Big Data en Entornos Seguros**

**Creación de una IA optimizada
para la asignatura de
Fundamentos de Programación**

Presentado por Víctor Hernando Aragón
en Universidad de Burgos — 3 de febrero
de 2026

Tutor: Carlos E. Vivaracho Pascual

Universidades de Burgos, León y Valladolid



Máster universitario en Inteligencia de Negocio y Big Data en Entornos Seguros

D. Carlos E. Vivaracho Pascual, profesor del departamento de nombre Ingeniería Informática de la Universidad de Valladolid.

Expone:

Que el alumno D. Víctor Hernando Aragón, con DNI 71233566J, ha realizado el Trabajo final de Máster en Inteligencia de Negocio y Big Data en Entornos Seguros titulado Creación de una IA optimizada para la asignatura de Fundamentos de Programación.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 3 de febrero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Carlos E. Vivaracho Pascual

D. Alma María Pisabarro Marron

Resumen

Este proyecto abarca la necesidad de la creación de un modelo de inteligencia artificial open source el cual esté adaptado a las necesidades de la asignatura de Fundamentos de Programación impartida en el grado de Ingeniería informática de la universidad de Valladolid a través de un reentrenamiento con el código en Java y normas impartidas en la asignatura.

El objetivo es un modelo de IA que este adaptado al contexto de la asignatura que están cursando, no como los modelos que normalmente suelen ser consultados, de esta manera obtendrán respuestas y resolverán las dudas adaptadas a los contenidos de la asignatura.

El proyecto será desarrollado en el lenguaje de programación de Python y con el hardware de la escuela de ingeniería informática de Valladolid.

Descriptores

Inteligencia artificial, Fundamentos de Programación, Java, LLM.

Abstract

This project covers the need for the creation of an open source artificial intelligence model which is adapted to the needs of the Fundamentals of Programming course taught in the degree of Computer Engineering at the University of Valladolid through a retraining with the Java code and standards taught in the course.

This project will allow students to use an AI model that is adapted to the context of the subject they are taking, not like the models that are usually consulted, so they will get answers and solve the doubts adapted to the contents of the subject.

The project will be developed in the Python programming language and with the hardware of the school of computer engineering of Valladolid.

Keywords

Artificial intelligence, Fundamentals of Programming, Java, LLM

Índice general

Índice general	iii
Índice de figuras	vi
Índice de tablas	vii
Memoria	1
1. Introducción	3
1.1. Contexto	3
1.2. Objetivos	4
1.3. Nicho de mercado	4
1.4. Estructura de la memoria	5
2. Metodología y planificación	7
2.1. Metodología CRISP-DM	8
2.2. Planificación inicial del proyecto	9
2.3. Diagrama de Gantt inicial del proyecto	10
2.4. Plan de Riesgos	11
2.5. Planificación final del proyecto	14
2.6. Presupuesto	16
3. Análisis de requisitos	19
3.1. Requisitos de estilo y legibilidad	19
3.2. Requisitos de Estructura y Flujo de Control	20
3.3. Requisitos de Arquitectura y Diseño Modular	21

3.4. Restricciones Algoritmicas	21
4. Investigación y conceptos teóricos	23
4.1. Conceptos básicos	23
4.2. Formas de adaptación de un modelo al contexto	24
5. Alternativas de diferentes IA´s	33
5.1. Descripción del problema	33
5.2. Búsqueda de modelos	34
6. Selección de modelo y proceso de implementación	39
6.1. Pruebas realizadas sobre los modelos	39
6.2. Implementación inicial	39
6.3. Reemplazo del modelo	41
7. Dataset para reentrenar el modelo y frontal	43
7.1. Obtencion de los datos de entrenamiento	43
7.2. Estructura del conjunto de entrenamiento	44
8. Evaluación del modelo y discusión	47
8.1. Evaluación del modelo	47
8.2. Discusión rendimiento del modelo	51
9. Tecnologías utilizadas	53
9.1. Python	53
9.2. Unsloth	53
9.3. Discord	54
9.4. GanttProject	54
9.5. Overleaf	54
9.6. Modelo de gran lenguaje de Meta	54
9.7. Openwebui	55
9.8. HuggingFace	55
10. Conclusiones y Líneas de trabajo futuras	57
10.1. Conclusiones generales	57
10.2. Limitaciones encontradas	58
10.3. Lineas de trabajo futuras	58
Apéndices	60
Apéndice A Malas prácticas en programación	63

<i>Índice general</i>	v
Apéndice B Encuesta Expertos	71
Apéndice C Documentación de usuario	75
C.1. Introducción	75
C.2. Requisitos de usuarios	75
C.3. Instalación	75
C.4. Manual del usuario	76
Bibliografía	79

Índice de figuras

2.1.	Foto de la metodología CRISP-DM	7
2.2.	Foto del diagrama de gant previsto al iniciar el proyecto[6] . . .	11
2.3.	Foto del diagrama de gant previsto al iniciar el proyecto[6] . . .	16
5.4.	Grafico de comparación del rendimiento con respecto otros modelos open source basados en código [4]	35
7.5.	Imagen de la interfaz que emplea openwebui [18]	46
8.6.	Prompt realizado por el usuario y respuesta del modelo [18] . .	48
8.7.	Prompt realizado por el usuario y respuesta del modelo [18] . .	49
C.1.	Foto del terminal indicando los modelos existentes en ollama [18]	76
C.2.	Foto del frontal la primera vez que iniciamos sesión [18]	77

Índice de tablas

2.1. Planificación inicial del proyecto	10
2.2. Riesgo 1	12
2.3. Riesgo 2	12
2.4. Riesgo 3	13
2.5. Riesgo 4	13
2.6. Riesgo 5	14
2.7. Planificación inicial del proyecto	15
2.8. Coste total del proyecto	17
8.9. Prueba resolución de ejercicios modelo	47
8.10. Prueba evaluación de código correcto por parte del modelo	48
8.11. Prueba evaluación de código con error de mala praxis	49
8.12. Prueba evaluación de respuesta ejercicio corregido	50
8.13. Prueba evaluación de detección de punto de rotura de la estruc- turas de control	50
8.14. Prueba evaluación de detección de bucles infinitos	51

Memoria

Introducción

1.1. Contexto

Los avances de los últimos años en Inteligencia Artificial generativa y Grandes Modelos de lenguaje (LLM's) han llevado a la creación de aplicaciones de ámbito general (e.g., ChatGPT, CoPilot, DeepSeek) que actualmente son utilizadas por la mayoría de estudiantes para resolver dudas y ampliar los conocimientos de sus asignaturas.

Sin embargo, sus modelos carecen del contexto del alumno o de los conocimientos que son impartidos en las asignaturas que se usan, ya que han sido entrenados a través de documentación genérica de internet. Así, los resultados que ofrece para preguntas relacionadas con algunas asignaturas no están contextualizadas, no siempre son precisas y/o no son correctas, resultando en un aprendizaje negativo para los estudiantes.

Fundamentos de Programación es una asignatura del Grado de Ingeniería Informática que se encuentra en el primer cuatrimestre del primer año de carrera, donde típicamente sucede este problema, en el que personas que pueden haber o no empezado en el mundo de la programación por lo que es esencial aprender los conceptos correctamente.

La asignatura hace especial hincapié en las buenas prácticas de programación utilizando el lenguaje de programación Java y el paradigma de programación estructurada. Sin embargo, cuando se pregunta a estas aplicaciones sobre determinados ejercicios de programación, sus respuestas no están contextualizadas al paradigma utilizado. Por esta razón, se quiere adaptar un modelo de IA que sea capaz de indicar a los alumnos los errores que tienen en sus programas y plantear alternativas correctas a los mismos. No se busca que el sistema, como cualquiera de los actuales, dé una solución

a un problema, sino que la solución esté contextualizada. Si no es correcta, le puede indicar dónde y por qué y/o proponer soluciones correctas.

1.2. Objetivos

Como se ha comentado en la introducción, el principal objetivo de este proyecto es la creación de un modelo basado en una IA openSource que permita a los alumnos enviar preguntas de texto así como código en el lenguaje de programación Java principalmente para comprobar si el código introducido es correcto según el contexto de la asignatura de Fundamentos de Programación impartida en el grado de Ingeniería Informática de la Universidad de Valladolid en el primer cuatrimestre del primer año de carrera. Con ella los alumnos serán capaces de obtener los resultados esperados tanto a nivel de programación como aprendizaje, asentando los conceptos básicos impartidos en esta asignatura.

A parte de este que es el objetivo principal para llegar a el deberemos de cumplir los siguientes objetivos:

- Realizar un análisis inicial de los principales modelos de I.A open source, para estudiar cual de ellos se adapta mejor a las necesidades del proyecto.
- Contrastar las diferentes formas que existen en la actualidad para adaptar el modelo a la funcionalidad que irá destinado.
- Hacer un análisis de los recursos hardware que serán necesarios para la adaptación del modelo.
- Investigar e implementar las tecnologías necesarias para llevar a cabo la adaptación del modelo.
- Realizar diferentes pruebas para ver si el rendimiento del proyecto es correcto y las respuestas ofrecidas satisfacen las necesidades del proyecto

1.3. Nicho de mercado

Al tratarse de un proyecto totalmente personalizado para los requisitos de un cliente concreto, que en este caso es mi tutor, no ha sido necesario realizar un estudio de mercado.

En cuanto a los usuarios objetivo, la intención del cliente es que el simulador sea empleado por los alumnos de la Universidad de Valladolid en la Escuela de Ingeniería Informática en la asignatura de Fundamentos de Programación, asignatura obligatoria del 1º curso.

1.4. Estructura de la memoria

Este documento se estructura de la siguiente forma:

Capítulo 1 Introducción: En este capítulo se describe el contexto y motivación a través de los cuales el proyecto empezó a realizarse, así como los objetivos principales que debe de cumplir el proyecto y el nicho de mercado al que va destinado.

Capítulo 2 Metodología y planificación: En este capítulo se describe la metodología usada durante el desarrollo del proyecto, el análisis y plan de riesgos, así como un análisis de costes y seguimiento del proyecto

Capítulo 3 Análisis de requisitos : En este capítulo se describen los requisitos que debe de cumplir el modelo.

Capítulo 4 Investigación y conceptos teóricos : En este capítulo se describen los principales conceptos que han sido necesarios para llevar a cabo la realización de este proyecto y su entendimiento.

Capítulo 5 Alternativas de diferentes IA's : En este capítulo se describe el estudio que se realizó de diferentes modelos de inteligencia artificial con el fin de que cumplieran con el propósito del proyecto y sus objetivos.

Capítulo 6 Selección de modelo y proceso de implementación : En este capítulo se describe la selección final del modelo escogido para reentrenarlo y su implementación.

Capítulo 7 Dataset de reentrenamiento y frontal : En este capítulo se habla del conjunto de datos empleado para reentrenar el modelo, que tipo de formato se ha seguido así como ejemplos del mismo, además se añade la tecnología empleada para la creación de la interfaz a través de la que los usuarios harán preguntas al modelo.

Capítulo 8 Evaluación del modelo y discusión : En este capítulo se desglosa y expone el cuestionario rellenado por los expertos que han probado el modelo y su valoración en los diferentes aspectos que debe de realizar.

Capítulo 9 Tecnologías utilizadas : En este capítulo se habla de las tecnologías software empleadas para la realización del proyecto.

Capítulo 10 Conclusiones y líneas de trabajo futuras : En este capítulo se habla de las conclusiones tras la realización del proyecto, las posibles mejoras a realizar y trabajos a futuro.

Metodología y planificación

Para el desarrollo de este proyecto se ha empleado la metodología Cross-Industry Standard Process for Data Mining (CRISP-DM), ya que se adapta perfectamente al desarrollo realizado en este proyecto de investigación en el cual tras comprender los requisitos y como debería ser el conjunto de datos para el reentrenamiento se ha generado y creado el modelo reentrenado sobre el que se han realizado varias iteraciones hasta conseguir un modelo con unos buenos resultados para poder ser usado.



Figura 2.1: Foto de la metodología CRISP-DM

2.1. Metodología CRISP-DM

Los orígenes de esta metodología [13] vienen del teórico de la informática Winston W. Royce, quien en 1970 elaboró su ensayo *Managing the Development of Large Software Systems*, donde proponía que el modelo en cascada se efectuara de manera iterativa. Mas tarde, a partir de 1985, este modelo se hizo famoso en el desarrollo del software cuando el Departamento de Defensa de los Estados Unidos publicó el Estándar 2167 para el desarrollo de software militar, el cual era una variante de su modelo, denominado cascada rígida.

A la hora de emplear la metodología en cascada[12] tenemos que tener en cuenta sus ventajas y desventajas. Sus principales ventajas son que tiene una estructura clara y permite transmitir al cliente la información de en qué paso del desarrollo nos encontramos exactamente. Además, la estructura en cascada también permite la realización de las fases en paralelo o el uso de prototipos, según el tipo de proyecto. En cuanto a sus desventajas, las principales son que dificulta la modificación de requisitos durante el desarrollo, así como que la realización de pruebas se debe realizar al final del desarrollo. Sin embargo, este último punto en nuestro caso tiene poca importancia, ya que este proyecto requiere que su implementación sea al menos parcialmente funcional para poder ser probado.

Esta metodología se empleo en la primera fase del proyecto en la cual se realizó una investigación de que modelos de IA serían los adecuados para la tarea, que tipo de entrenamiento se realizaría así como se realizaría ese entrenamiento, todas estas fases se han realizado de manera secuencial de tal manera que cuando tenía una de ellas se pasaba a la siguiente, siguiendo el modelo en cascada.

El principal motivo de elección de este modelo de gestión del proyecto es debido a la índole de investigación que ha requerido para desarrollar las primeras fases del proyecto y poder tener el flujo completo funcionando. Lo cual ha requerido no solo investigación por mi parte sino contacto con la Universidad de Valladolid para el uso de recursos que permitan el entrenamiento de un modelo de IA de una manera optima.

Fases del Proyecto

Una vez que hemos decido emplear la metodología en cascada distinguiremos las siguientes fases a lo largo del desarrollo del proyecto:

- Documentación: en esta fase se ha dedicado a comprender como poder realizar un reentrenamiento de un modelo de IA, diferentes formas de entrenamiento así como los componentes de los propios LLM, asentando los conceptos necesarios para realizarlo.
- Investigación: tras entender la parte teórica, se ha procedido a realizar una investigación de aquellos modelos de IA open source que pueden servir para la realización del proyecto, así como las herramientas que nos permitirán realizar la tarea de reentrenamiento y los recursos hardware como software necesarios para la tarea.
- Análisis: tras obtener varios posibles candidatos entre los modelos de IA se ha realizado un estudio en profundidad de los mismos con el fin de obtener el modelo que mejor se ajusta a la tarea de este proyecto
- Diseño, implementación: tras obtener el modelo adecuado y las herramientas para poder realizar el entrenamiento se ha realizado su implementación. Este punto ha sido en concreto donde se ha aplicado una metodología de ensayo error. En el cual se han realizado un ciclo de prueba, evaluación y ajuste hasta obtener el resultado que queremos del modelo.
- Pruebas de rendimiento del modelo: tras realizar su reentrenamiento se ha pasado el modelo a través de diferentes métricas y evaluaciones con el fin de ver si el modelo reentrenado cumple con las necesidades del proyecto, en el caso de no ser así se han realizado tareas de ajuste sobre los parámetros para obtener la configuración mas optima del mismo
- Finalización de la documentación del proyecto: se han realizado las últimas modificaciones al documento con el fin de la mejora de esta memoria de cara a su presentación.

2.2. Planificación inicial del proyecto

Empleando el modelo en cascada visto en el apartado anterior, se elaboró al inicio del proyecto una planificación en la cual abarcamos sus distintas fases. Abarcando las 225 horas que aproximadamente se corresponden con los 9 créditos designados al proyecto.

La Tabla 2.1 muestra las fechas establecidas para cada fase del TFG.

Tarea	Duración	Comienzo	Fin
INVESTIGACIÓN			
Investigación	100 horas	25/06/25	10/07/25
ANÁLISIS			
Análisis de los requisitos del proyecto	10 horas	11/07/25	14/07/25
DISEÑO, IMPLEMENTACIÓN Y PRUEBAS			
Implementación del modelo base	20 horas	15/07/2025	31/07/25
Implementación del framework para entrenar el modelo	20 horas	01/08/2025	15/08/2
Evaluación sobre el modelo reentrenado	30 horas	16/08/2025	20/08/25
Realización del frontal	20 horas	21/08/2025	25/08/25
FINALIZACIÓN DEL PROYECTO			
Finalización de la documentación y entrega del TFM	25 horas	26/08/25	30/08/25

Tabla 2.1: Planificación inicial del proyecto

2.3. Diagrama de Gantt inicial del proyecto

En la Figura 2.2 podemos apreciar la duración prevista de cada una de las etapas del proyecto. En este diagrama aparecen las tareas principales especificadas anteriormente en la Tabla 2.1.

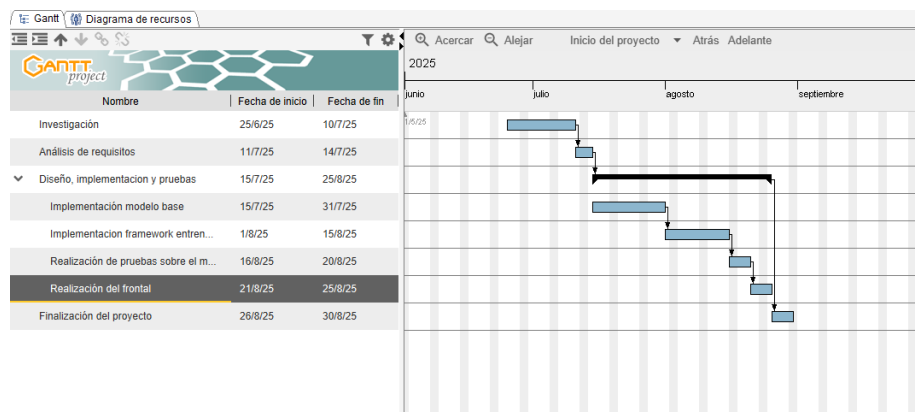


Figura 2.2: Foto del diagrama de gant previsto al iniciar el proyecto[6]

2.4. Plan de Riesgos

Para poder realizar un proyecto de cualquier índole es necesario elaborar un plan de riesgos. Estos riesgos deben de referirse a problemas futuros, los cuales podrían suceder durante el desarrollo del proyecto, y debe documentarse tanto la causa como el efecto que producirá el riesgo.

Para elaborar de manera correcta el plan de riesgos se han tenido en cuenta los siguientes puntos:

- La probabilidad de que se produzca el riesgo. Será medida a través de los valores: bajo, medio y alto.
- El impacto que tendrá el riesgo. Si llega a aparecer será medido a través de los valores: bajo, medio y alto.
- Plan de mitigación: conjunto de acciones que se deben realizar para reducir la probabilidad de que un riesgo ocurra.
- Plan de contingencia: conjunto de acciones que se deben realizar una vez el riesgo se ha ocurrido para reducir su impacto sobre el proyecto.

La Tabla 2.2 muestra el Riesgo 1, el cual es la estimación incorrecta del tiempo en el desarrollo del proyecto.

La Tabla 2.3 muestra el Riesgo 2, el cual es la perdida de tiempo de trabajo por ausencia del alumno, mientras que la Tabla 2.4 muestra el Riesgo 3, el cual representa el riesgo de ausencia del tutor.

La Tabla 2.5 muestra el Riesgo 4, el cual es la modificación de requisitos.

R1	Estimación incorrecta del tiempo
Descripción	El desarrollo del proyecto no se está llevando según las estimaciones iniciales de tiempo.
Probabilidad	Baja
Impacto	Medio
Plan de mitigación	■ Realizar una nueva estimación, para determinar el alcance real del proyecto. ■ Priorizar las tareas críticas.
Plan de contingencia	■ Definir el alcance del proyecto, acorde al tiempo disponible.

Tabla 2.2: Riesgo 1

R2	Ausencia del alumno por motivos de trabajo u enfermedad
Descripción	El estudiante no puede realizar el proyecto debido a otras responsabilidades laborales, familiares u enfermedad del mismo
Probabilidad	Alta
Impacto	Medio
Plan de mitigación	■ Comunicar a los tutores el posible retraso
Plan de contingencia	■ En función del tiempo perdido, redistribuir las horas o añadir un margen de tiempo a la planificación.

Tabla 2.3: Riesgo 2

R3	Ausencia de los tutores por motivos de trabajo u enfermedad
Descripción	El estudiante no puede realizar avances el proyecto por dependencia de contacto con los tutores del proyecto.
Probabilidad	Media
Impacto	Medio
Plan de mitigación	■ Realizar una comunicación vía email sobre la situación personal del tutor para que el alumno tenga constancia de ello
Plan de contingencia	■ Cancelar o aplazar las reuniones que se iban a tener con el alumno sobre el avance del proyecto. ■ Buscar un medio de comunicación alternativo, tales como reuniones por videoconferencia, para realizar las tutorías en el caso de que el tutor lo considere necesario.

Tabla 2.4: Riesgo 3

R4	Cambio en los requisitos del proyecto
Descripción	Se produce una modificación de los requisitos acordados al inicio del proyecto.
Probabilidad	Baja
Impacto	Alto
Plan de mitigación	■ Mantener informado al tutor del proyecto de problemas que surjan y el avance del proyecto de manera recurrente.
Plan de contingencia	■ Añadir un margen a la planificación del proyecto.

Tabla 2.5: Riesgo 4

La Tabla 2.6 muestra el Riesgo 5, el cual es la avería del ordenador de trabajo o caída de los servicios utilizados.

R5	Fallos en las aplicaciones o hardware para desarrollar el proyecto
Descripción	Se produce algún problema con el hardware o con los servicios software empleados.
Probabilidad	Media
Impacto	Alto
Plan de mitigación	■ Mantener los repositorios donde se almacena el proyecto lo más actualizados posible. ■ Tener una copia local del proyecto en nuestro ordenador.
Plan de contingencia	■ En el caso del ordenador se nos estropeara tendríamos que intentar emplear otro, si es posible. ■ Si alguno de los servicios software empleados dejara de funcionar, lo ideal sería hacer los cambios en local y esperar a que el servicio vuelva a estar disponible.

Tabla 2.6: Riesgo 5

2.5. Planificación final del proyecto

Tras haber pasado por todas las fases descritas en la planificación inicial como se puede apreciar en la Tabla 2.1, algunos de los riesgos anteriormente mencionados se han producido, concretamente:

Durante el proyecto se llegaron a activar los riesgos que aparecen en las tablas 2.3, 2.4 y 2.6 lo que produjo que se aplicaran las medidas de contingencia correspondientes a esos riesgos.

A continuación en la Tabla 2.7 muestra las fechas establecidas para cada fase del tfm después de la aparición de los riesgos y como afectaron la aplicación de los planes de contingencia al desarrollo del proyecto.

Como podemos apreciar de las 225 horas planeadas al inicio del proyecto se han elevado a un total de 314 horas las fases con mayor incremento han sido la de investigación y de pruebas del modelo esto se debe principalmente a los siguientes motivos:

Tarea	Duración	Comienzo	Fin
INVESTIGACIÓN			
Investigación	120 horas	25/06/25	14/08/25
ANÁLISIS			
Análisis de los requisitos del proyecto	20 horas	15/08/25	20/08/25
DISEÑO, IMPLEMENTACIÓN Y PRUEBAS			
Implementación del modelo base	20 horas	21/08/2025	01/09/25
Implementación del framework para entrenar el modelo	37 horas	02/09/2025	11/10/25
Evaluación sobre el modelo reentrenado	62 horas	12/10/25	08/12/25
Realización del frontal	20 horas	09/12/25	15/12/25
FINALIZACIÓN DEL PROYECTO			
Finalización de la documentación y entrega del TFM	35 horas	16/12/25	11/01/26

Tabla 2.7: Planificación inicial del proyecto

- La fase de investigación se alargó mas de lo esperado debido a que fue difícil encontrar un framework que permitiera realizar las tareas de re entrenamiento de modelos de manera sencilla realizando múltiples investigaciones de diferentes proyectos y alternativas tanto de implementación solo con librerías básicas o frameworks hasta dar con el correcto. El cual se emplearía en la implementación final.
- La fase de realización de pruebas sobre el modelo reentrenado se ha alargado debido a que ha resultado difícil conseguir un buen resultado del modelo reentrenado, debido al problema de la falta de datos para el entrenamiento así como que el modelo inicial no fue el adecuado a la hora de hacer la inferencia.
- A todo esto se suma la realización del proyecto junto con el trabajo del alumno a jornada completa lo que no ha permitido ser del todo consistente con los tiempos estipulados al inicio del proyecto.

La Figura 2.3 muestra el diagrama de Gantt final del proyecto, donde se pueden apreciar la duración de cada una de las etapas una vez finalizado el proyecto. En el diagrama se pueden observar las tareas principales del proyecto.

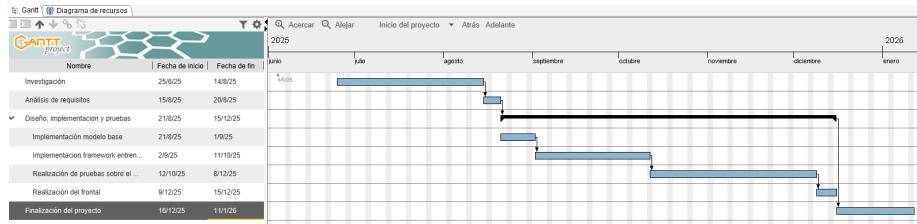


Figura 2.3: Foto del diagrama de gant previsto al iniciar el proyecto[6]

2.6. Presupuesto

En todos los proyectos existe un presupuesto, en el cual se muestran los gastos del proyecto, teniendo en cuenta las herramientas hardware y software utilizadas, así como el personal que ha trabajado en este proyecto.

Presupuesto Software

El proyecto se ha realizado en su mayoría con herramientas software gratuitas.

En el caso de que el proyecto se estuviera desarrollando en una empresa esta debía de asumir el coste empresarial de Unsloth [20] herramienta empleada para realizar el reentrenamiento del modelo, su coste supone un total de 50€ al mes que durando el proyecto alrededor de 7 meses supondría un coste total de 350€ a fecha de 10/12/2025

Presupuesto Hardware

El hardware son los dispositivos que han sido empleados para la realización del proyecto, los cuales son:

- Maquina virtual Linux: 70€ mes 490€ en total
- Monitor Acer: tiene un precio de 150€
- Tarjeta gráfica NVIDIA A40: 7104,52€

Presupuesto recursos humanos e infraestructura

En cuanto al coste de personal, considerando al alumno un desarrollador Mid-Serior de tres años, el sueldo oscila alrededor de los 33.000 € anuales en contratos a jornada completa, es decir, 40 horas a la semana, o unas 160 horas al mes, aplicando un IRPF del 17,11 % sobre el salario bruto. El sueldo mensual equivalente serían 1.800,94 € en 14 pagas netos al mes. Como tal el gasto a la empresa por horas partiendo del total bruto en 7 meses son 19.250 €

Sobre el presupuesto de infraestructura, teniendo en cuenta que el trabajo se ha realizado en la vivienda personal del alumno, podemos considerar el coste de internet como gasto. Teniendo en cuenta que la tarifa son 44 € al mes y se ha llevado a cabo el proyecto durante 7 meses el coste total es de 308 €

Presupuesto total

En la Tabla 2.8 procedemos a sumar las cantidades de cada uno de los apartados anteriores para obtener el total.

Concepto	Coste
Presupuesto software	350 €
Presupuesto hardware	7.744,22 €
Presupuesto recursos humanos e infraestructura	19.558 €
Coste total	27.652,22 €

Tabla 2.8: Coste total del proyecto

Análisis de requisitos

En este apartado se hablará de los diferentes requisitos que debe cumplir el modelo de I.A generado para la asignatura de FPROG con el fin de que sea util tanto para los alumnos como para los profesores.

Este apartado de requisitos se ha elaborado a partir del documento de malas praxis incluido en el apendice [A](#) Malas prácticas de programacion

3.1. Requisitos de estilo y legibilidad

El modelo deberá generar código que maximice la claridad y facilite la documentación.

RF-01 Convenciones de nombrado

El código generado por el modelo debe utilizar identificadores que sugieran su propósito.

- Debe mantener consistencia (ej. camelCase) y respetar las convenciones de Java (clases con mayúscula, variables/métodos con minúscula, constantes en mayúsculas).
- Se deben evitar nombres excesivamente largos, excepto en fórmulas conocidas o convenciones como son el nombre de los índices en los bucles (i,j)

RF-02: Gestión de Constantes

El modelo no debe permitir el uso de valores literales directos en el código también conocidos como constantes mágicas.

- este deberá refactorizar estos valores asignándoles un nombre significativo (definición global o de clase).
- Deberá de permitir ciertas excepciones como: 0 y 1 en bucles, 2 para paridad, 100 para porcentajes, potencias de 10 en sistema métrico y valores universales de tiempo/ángulos (60, 24), números o letras de elección de opción, mensajes de salida, “Si”, “No” entre otros,

RF-03: Documentación y Organización

El modelo debe evitar comentarios redundantes o inadecuados, utilizándolos solo cuando aporten valor a la documentación interna. Así como el código debe estar bien indentado y organizado, recomendándose métodos que no superen una página y líneas de menos de 80 caracteres.

3.2. Requisitos de Estructura y Flujo de Control

El modelo debe garantizar el uso correcto de las estructuras lógicas, evitando flujos que no sean adecuados según lo visto en la asignatura.

RF-04: Integridad de Estructuras de Control

El modelo no debe generar sentencias *break* para salir de bucles ni sentencias *return* que no sean la última instrucción de la secuencia o método. El uso de banderas booleanas es preferible si es necesario.

También debe de saber cuando distinguir entre el uso de los iteradores *for*, *while*, *do-while*

RF-05: Integridad de los bucles

El modelo debe de saber identificar cuando ha sido modificada la variable de control o los límites de un bucle *for* dentro de su cuerpo, ya que hace el comportamiento impredecible. Así como evitar la secuencia *loop-switch*

(un if dentro de un bucle para tratar un índice especial). En su lugar, debe dividir el bucle en partes secuenciales.

RF-06: Selección de Alternativas

El modelo debe elegir lógicamente entre secuencias y estructuras alternativas (*if/else*). No se debe usar una secuencia de if independientes cuando la lógica requiere exclusión mutua (*else if*).

3.3. Requisitos de Arquitectura y Diseño Modular

El código generado debe mostrar un diseño modular robusto, evitando dependencias ocultas o acoplamiento excesivo.

RF-07: Prohibición de Variables Globales

El modelo no debe declarar variables en la generación de código fuera de los métodos para compartir datos entre ellos, ya que dificulta la reutilización y el mantenimiento. Así como identificarlo como una mala praxis.

RF-08: Gestión de Parámetros

El modelo debe de identificar que los métodos deben comunicarse mediante paso de parámetros.

RF-09: Cohesión y Desacoplamiento

El modelo debe detectar y eliminar fragmentos de código que nunca se ejecutan ("dead code") y refactorizar líneas repetidas para evitar duplicación. Así como detectar el uso de un "falso main" que delegue todo el trabajo a una sola llamada de método; debe orquestrar el flujo.

3.4. Restricciones Algorítmicas

RF-10: Uso de Recursividad

El modelo debe priorizar soluciones iterativas. La recursividad solo se permitirá en problemas donde sea la solución natural. Se prohíbe explícitamente la recursión final (tail recursion) si encubre una iteración simple.

Investigación y conceptos teóricos

En este capítulo nos centraremos especialmente en conceptos teóricos que han tenido que ser investigados, estudiados y comprendidos por el alumno, de tal manera de poder llegar así a la solución planteada.

4.1. Conceptos básicos

- Inteligencia Artificial (IA): Según la comisión europea son sistemas de software (y posiblemente también de hardware) diseñados por humanos que, ante un objetivo complejo, actúan en la dimensión física o digital percibiendo su entorno, a través de la adquisición e interpretación de datos estructurados o no estructurados y razonando sobre el conocimiento, procesando la información derivada de estos datos y decidiendo las mejores acciones para lograr el objetivo dado. [\[7\]](#)
- Modelo extenso de lenguaje (LLM): se trata de modelos que son entrenados con grandes cantidades de texto proveniente de artículos, dentro de estos podemos diferenciar entre los LLM tradicionales, aquellos que han sido entrenados únicamente con texto como por ejemplo GPT-3, LLaMA y los LLM Multimodales que han sido entrenados con otras fuentes de información que no es solo texto, como imágenes, video y audio.
- Parámetros del modelo: matrices numéricas que ajustan cómo el modelo transforma los datos de entrada en salida, a mayor número de parámetros mayor capacidad de aprendizaje y comprensión de patro-

nes, pero también supone un mayor consumo de memoria y coste computacional

- Tokens: Los tokens son palabras, juegos de caracteres o combinaciones de palabras y puntuación que generan los modelos de lenguaje grandes (LLM) cuando descomponen el texto. La tokenización es el primer paso del entrenamiento.
- Dataset de entrenamiento: conjunto de datos con cierta estructura que es capaz de entender el modelo con el fin de reentrenarlo para la tarea específica de los datos. .

4.2. Formas de adaptación de un modelo al contexto

Este apartado pretende recoger las diferentes formas de conseguir que el modelo seleccionado emplee el conjunto de datos basado en un conjunto de programas java. Existen diferentes formas de entrenar a nuestro modelo entre las cuales podemos destacar son RAG (Retrieval-Augmented Generation), LoRA/QLoRA (Low-Rank Adaptation/ Quantum Low-Rank Adaptation) y el fine-tuning completo (ajuste fino entrenando todos los parámetros del modelo). A continuación, para cada técnica se explica como funciona, las herramientas típicas para implementarla, requisitos de hardware, ventajas, desventajas y escenarios recomendados de uso.

RAG (Retrieval-Augmented Generation)

RAG (Generación Aumentada con Recuperación de información) es un método que no altera los parámetros del modelo base, sino que lo potencia proporcionándole información adicional relevante extraída de una base de conocimiento externa en cada consulta. RAG extiende las capacidades del LLM hacia un dominio específico o base de conocimiento interna sin necesidad de reentrenar el modelo. Es una forma rentable de mantener las respuestas del modelo actualizadas, precisas y enfocadas al contexto deseado, porque aprovecha información fuera de los datos originales de entrenamiento del modelo.^[2]

Funcionamiento

Con RAG, el proceso de inferencia del modelo se complementa con un buscador que permite recuperar la información actuando antes de la genera-

ción de la respuesta. Cuando el usuario realiza una consulta o prompt, este componente de búsqueda utiliza la entrada para recuperar datos relevantes de una fuente externa (por ejemplo, un conjunto de documentos, una base de código o una base de conocimientos)[2].

Los datos externos (textos, fragmentos de código, documentación, etc.) suelen haber sido previamente procesados con modelos de embeddings para convertirlos en representaciones numéricas, almacenándose en una base de datos vectorial que permite búsquedas semánticas eficientes[2].

Una vez obtenidos los documentos o fragmentos más relevantes mediante la búsqueda (por similitud vectorial, coincidencia semántica, etc.), se aumenta la entrada original del usuario añadiendo ese contenido recuperado al prompt. Finalmente, el modelo de lenguaje genera su respuesta utilizando tanto su conocimiento entrenado como la información específica proporcionada en el prompt [2].

En esencia, RAG inyecta conocimiento puntual de la base externa en la entrada del modelo para guiar la generación de una respuesta más informada. Diagrama conceptual de RAG: el modelo de lenguaje (LLM) recibe la consulta del usuario junto con datos relevantes recuperados de una base de conocimiento externa, utilizando ambos para generar una respuesta precisa. El flujo típico es:

1. Convertir la pregunta del usuario en una consulta semántica
2. Buscar en la base de conocimiento vectorial los documentos o fragmentos más relacionados
3. Añadir esos fragmentos al prompt antes de la generación

De este modo, no se modifica el modelo original, sino que se le proporciona contexto adicional en tiempo de inferencia.

Requisitos Hardware

En comparación de los otros métodos de adaptación de modelos que veremos más adelante se trata realmente de un costo computacional bajo, ya que como hemos visto realmente no realizamos un entrenamiento del modelo sino que realizamos la búsqueda del prompt inicialmente en la base de conocimiento, por lo que el hardware principalmente será necesario para almacenar el modelo original y mantener la estructura de búsqueda sobre la base de conocimiento y puede ejecutarse en un entorno doméstico siempre que el modelo por ejemplo no tenga muchos parámetros

Ventajas

- No altera el modelo base: No hay que reentrenar el LLM, lo que evita costos de cómputo elevados y riesgos de degradar el modelo. Se aprovecha el conocimiento general del modelo y sólo se le inyecta información específica cuando hace falta.
- Información actualizada y específica: Permite al modelo acceder a datos más recientes o especializados que no estaban en su entrenamiento original, así como modificar la base de conocimiento para mantener actualizadas las respuestas que de el LLM.
- Control y trazabilidad: podemos controlar la fuente de la información que el modelo usa, imitándola a bases de conocimiento específicas.

Desventajas

- Dependencia del motor de búsqueda: La calidad de las respuestas depende en gran medida de qué tan bien funcione el módulo de búsqueda. Si la base de datos vectorial no devuelve los fragmentos relevantes o está incompleta, el modelo puede seguir alucinando o dando información incorrecta. Requiere por tanto un buen diseño de la base de conocimiento y afinamiento del proceso de búsqueda.
- Complejidad del sistema: Comparado con usar sólo un LLM fine-tuneado, RAG introduce componentes adicionales, lo cual puede ser más complejo de implementar y mantener. Hay más partes móviles que pueden fallar o añadir latencia (por ejemplo, la búsqueda vectorial añade pasos extra en cada consulta).
- No estamos entrenando el modelo: realmente no estamos adaptando el modelo a nuestras necesidades, simplemente el prompt que nos llega por parte del usuario lo consultamos en la base de conocimiento y este luego se pasa al modelo.

Casos de uso

- Información dinámica o actualizable: Si el conocimiento del dominio cambia con frecuencia (nuevas versiones de software, noticias al día, datos en tiempo real), RAG permite mantener las respuestas actualizadas simplemente actualizando la base de conocimiento, sin tener que re-entrenar el modelo constantemente.

- Recursos de cómputo limitados: Cuando no se cuenta con GPUs potentes ni tiempo para entrenamiento, RAG ofrece una forma de adaptar un modelo grande a un contexto específico aprovechando la inferencia solamente. Por ejemplo, empresas que quieran personalizar un chatbot con sus documentos internos, pero no puedan costear entrenar de nuevo un LLM, pueden usar RAG para lograrlo de forma eficiente.
- Cuando el modelo necesita contexto para un caso de uso específico: si ya empleamos de base un buen modelo para la tarea concreta, como por ejemplo un modelo el cual esta centrado en código y le damos una base de conocimiento mas concreta entonces obtendremos unas buenas respuestas y encima adaptadas al contexto específico.

LoRA / QLoRA (Low-Rank Adaptation)

LoRA (Low-Rank Adaptation) es una técnica de ajuste fino eficiente en parámetros que permite adaptar grandes modelos con un costo computacional muy bajo en comparación con el fine-tuning tradicional. La idea central es congelar los pesos originales del modelo y añadir un pequeño conjunto de parámetros entrenables adicionales que aprenden la tarea o el dominio nuevos. Estos parámetros adicionales toman la forma de adaptadores de rango bajo (low-rank), de ahí el nombre LoRA. Durante el entrenamiento, únicamente se ajustan estos adaptadores (que representan un porcentaje muy pequeño del total de parámetros), manteniendo intactos los pesos originales. Al término, el modelo utiliza los pesos originales más las pequeñas modificaciones aprendidas para generar respuestas especializadas. QLoRA es una variante de LoRA que combina esta idea con cuantización a 4 bits del modelo base para minimizar aún más el uso de memoria. En QLoRA, el modelo preentrenado se carga en formato cuantizado de 4 bits (en lugar de 16 o 32 bits típicos), se mantiene congelado, y se entrenan igualmente adaptadores LoRA encima. Esto permite afinar modelos extraordinariamente grandes usando hardware relativamente accesible, sin pérdida significativa de calidad.[3]

Funcionamiento

En LoRA estándar, las matrices de pesos de determinadas capas del modelo se complementan con unas matrices de bajo rango que se inicializan de forma que inicialmente no afecten la salida. Durante el entrenamiento, en lugar de actualizar la gran matriz de pesos original W (que permanece

congelada), se entrenan dos matrices mucho más pequeñas, A y B , cuya multiplicación produce una aproximación de bajo rango de las posibles actualizaciones a W . En concreto, W se mantiene fijo y la actualización efectiva es $W + \Delta W$, donde $\Delta W = A \times B$ con A de dimensión $(n \times r)$ y B de $(r \times m)$ siendo r el rango bajo elegido (mucho menor que n o m). De esta forma, el número de parámetros ajustables es minúsculo comparado con W (aproximadamente $(n + m)r$, en lugar de nm). Al finalizar el entrenamiento, el modelo puede usar $W + \Delta W$ como pesos efectivos.[3]

Para resumir, esto significa que sólo se aprenden las diferencias necesarias para la nueva tarea o dominio, sin alterar los conocimientos originales del modelo. En QLoRA, se agrega un paso previo: cuantizar el modelo base a 4 bits.[21] El modelo preentrenado en 16 bits (FP16) se convierte a un formato de 4 bits (normalmente NF4, Normalized Float 4, óptimo para pesos distribuidos normalmente). Luego, ese modelo 4-bit se congela y se incorporan los adaptadores LoRA como en el método original.

Durante el entrenamiento, se propagan los gradientes a través del modelo cuantizado congelado hacia los adaptadores, actualizando sólo los pesos de LoRA. La cuantización reduce drásticamente la memoria necesaria para almacenar el modelo, y al no actualizar los pesos cuantizados, se evita degradar su valor original con operaciones de entrenamiento. Para el cálculo de gradientes, QLoRA de hecho decuantiza temporalmente los pesos a 16 bits en el pase hacia atrás, pero sólo calcula gradientes para los parámetros LoRA[21]

En suma, QLoRA conserva las ventajas de LoRA (pocos parámetros a entrenar) sumándole una compresión del modelo base, logrando una eficiencia excepcional sin prácticamente sacrificar rendimiento (según estudios, QLoRA alcanza resultados a la par del fine-tuning completo en 16 bits en muchos casos).[21]

Requisitos Hardware

LoRA y especialmente QLoRA se diseñaron para reducir radicalmente los requerimientos de memoria y cómputo al adaptar grandes modelos. En la práctica, esto significa que se puede fine-tunear un modelo grande en una sola GPU de gama prosumidor o incluso en GPU de menor VRAM. Por ejemplo, se ha reportado que full fine-tuning de un modelo de 7B parámetros en FP16 requeriría del orden de 40–60 GB de VRAM, mientras que con LoRA ese mismo modelo puede entrenarse con 8 GB de VRAM, haciendo posible su uso en entornos domésticos.[?]

Ventajas

- Extremadamente eficiente en recursos: La principal ventaja es la drástica reducción en uso de VRAM y cómputo durante el entrenamiento. Sólo se ajusta una fracción ínfima de los parámetros del modelo, además de que obtiene resultados similares a un finetuning completo.
- Conserva el conocimiento original: Dado que el modelo base no se modifica, no hay riesgo de forgetting o de dañar las capacidades generales del LLM. El adaptador aprende sobre el dominio, pero el conocimiento original del modelo (sintaxis general, gramática, conocimientos del mundo) permanece intacto.
- Distribución y despliegue sencillos: Relacionado con lo anterior, compartir o desplegar un modelo adaptado con LoRA es más fácil, ya que sólo se comparten los pesos de los adaptadores (p. ej. vía HuggingFace Hub) que son archivos pequeños, y el destinatario los aplica al modelo base público. Esto facilita la colaboración y la iteración entre equipos, sin intercambiar enormes ficheros de modelo.

Desventajas

- Complejidad de integración en inferencia: Para usar un modelo LoRA, se necesita cargar tanto el modelo base como los pesos del adaptador añadiendo una capa extra de complejidad operacional a tener un modelo completamente ajustado.
- No entrena todo el potencial: Si bien LoRA adapta bien el modelo, no permite ajustar todos los matices ya que muchos parámetros quedan sin modificación, por lo que en conjuntos de datos grandes es más recomendable realizar un entrenamiento completo de todos los parámetros, ya que LoRA busca cambios en un subespacio limitado.
- Limitaciones de QLoRA específicas: En QLoRA, al usar cuantización 4-bit, actualmente no es posible realizar entrenamiento 4-bit en CPU, debe ser en GPU con CUDA.

Casos de uso

- Cuando los recursos son limitados pero se necesita especialización: LoRA/QLoRA es la técnica de elección cuando no se dispone de múltiples GPUs de gran memoria o clusters para entrenamiento.

- Proyectos de investigación o prototipos rápidos: Si se desea experimentar con adaptar modelos a distintas tareas de forma ágil.
- Múltiples dominios o tareas en un mismo modelo base: Cuando se quiera mantener un solo modelo base y tener variaciones especializadas para distintos casos como un mismo LLM base adaptado por separado a código Java, a generación de documentación técnica.
- Cuando se requiere conservar el modelo original intacto: Si por algún motivo necesitamos que el modelo pueda volver a su estado original o mantener la opción de usarlo en tareas generales, LoRA es ideal porque no modifica los pesos base.

Fine-tuning completo del modelo

El fine-tuning completo es el enfoque tradicional en el que se entrenan todos los parámetros del modelo de lenguaje en el conjunto de datos específico, ajustando así completamente el modelo a la nueva tarea o dominio. En este caso no se introduce arquitectura adicional: simplemente se toma el modelo preentrenado existente y se continúa entrenando con los datos proporcionados (normalmente con una tasa de aprendizaje baja para no destruir lo ya aprendido). Al finalizar, todos los pesos del modelo pueden haberse actualizado para reflejar los patrones del nuevo conjunto de datos. Este método convierte esencialmente al modelo generalista en un modelo especializado aprendiendo directamente de los datos específicos. Las principales desventajas son que es costoso a nivel de recursos y tiene sus riesgos de cara al modelo final.

Funcionamiento

Es similar a cuando se entrena el modelo original pero partiendo de los pesos ya aprendidos para los parámetros en pre-entrenamiento. Se formula un objetivo y se usa el algoritmo de optimización para ajustar los gradientes de todos los parámetros en cada paso, minimizando la función de pérdida en el conjunto de datos específico. Durante este proceso, el modelo puede adaptar todos sus niveles de representación: desde capas inferiores (aprendiendo algunas sintaxis específicas del código Java del conjunto de datos que estamos usando) hasta las superiores (quizás adoptando cierto estilo en las respuestas

Tras el fine-tuning, se obtiene un nuevo modelo (un nuevo conjunto completo de pesos) especializado en los datos proporcionados. En nuestro caso específico si afinamos completamente un modelo con un con un conjunto

de datos de código Java, el resultado es un modelo cuyo conocimiento del lenguaje Java y estará embebido en todos sus parámetros.

Requisitos Hardware

El fine-tuning completo de LLMs es notoriamente intensivo en recursos. Se necesita suficiente VRAM (memoria de GPU) para alojar el modelo completo y sus gradientes y los estados del optimizador, además de las activaciones durante el pase hacia adelante. Esto se acumula rápidamente: por ejemplo, un modelo de 7 mil millones de parámetros ocupa unos 14 GB en FP16 solo para los pesos; el optimizador requiere de mas recursos, y las activaciones/gradientes de una pasada pueden requerir otros 10 GB, llevando el total a on the order of 40–60 GB de VRAM para entrenar un modelo de 7B[?]

Ventajas

- Modelo autónomo especializado: El resultado del fine-tuning completo es un modelo especializado independiente, que no necesita componentes adicionales ni contexto externo para desempeñarse en el dominio. Toda la información relevante del dataset queda embebida en sus pesos.
- Adaptación de todos los pesos de los parámetros: Al actualizar todos los parámetros, el modelo tiene máxima capacidad de absorber el nuevo conocimiento o adaptar sus habilidades. No está limitado por un subespacio de ajustes (como en LoRA). Esto puede traducirse en un rendimiento ligeramente superior en la tarea específica cuando se dispone de suficientes datos.
- Personalización profunda del comportamiento: Más allá de conocimiento actual, al fine-tunear completo es posible cambiar comportamientos intrínsecos del modelo.

Desventajas

- Coste computacional alto: es de los tres métodos el que mayor coste computacional tiene tanto en hardware como en tiempo
- Riesgo de sobreajuste y forgetting: Al actualizarlo todo, existe el riesgo de sobreajustar al conjunto de datos específico, especialmente si este no es muy grande. El modelo puede terminar "memorizando" ^{en} exceso patrones o hasta ejemplos enteros del dataset o incluso el modelo

puede perder parte de su conocimiento general al sobrescribirlo con la nueva información.

- Necesidad de gran cantidad de datos de calidad: Para aprovechar un fine-tuning completo sin arruinar el modelo, se suele requerir un conjunto de datos amplio y bien adaptado al dominio.

Alternativas de diferentes IA´s

En este capítulo se centra principalmente en la investigación realizada para encontrar un modelo adecuado para la tarea del proyecto partiendo de un análisis inicial del problema hasta los posibles lenguajes candidatos.

5.1. Descripción del problema

Uno de los dos problemas a la hora de empezar a desarrollar la idea del TFM es encontrar el mejor modelo LLM que se adapte al caso de uso, teniendo en cuenta las características del problema:

Usuarios

Los usuarios que emplearán este modelo de IA con el fin de obtener respuesta a preguntas realizadas tanto en lenguaje natural como enviando código Java serán alumnos de 1º de carrera de la Universidad de Valladolid (UVA), por lo tanto, estamos hablando de personas que están empezando su etapa como ingenieros informáticos así como empezando con la programación, ya que existirán alumnos que, como fue mi caso, no han programado todavía antes de entrar a la carrera.

Asignatura

El modelo de IA está destinado concretamente para la asignatura de Fundamentos de Programación, que se imparte en el primer cuatrimestre del primer año de ingeniería informática. en la cual se enseñan las bases de la programación estructurada en el lenguaje de programación Java. Por lo que el modelo que tendremos que elegir será un modelo que esté preparado

o entrenado de base con un conjunto de datos de texto o información relacionada con los lenguajes de programación y específicamente con Java.

Tamaño del modelo

El modelo es necesario que no sea muy grande, ya que la tarea que desempeñará está bastante acotada por el contenido de la asignatura, ya que este modelo de IA que se escoja usará también datos obtenidos de programas Java facilitados por la Universidad de Valladolid con el fin de que las respuestas que devuelva el modelo estén basadas en los contenidos que se imparten en esta asignatura.

Pero si es importante que a pesar de que el tamaño del modelo este acotado se opte por un modelo el cual la principal fuente de datos con los que ha sido entrenado sea código, en concreto interesaría sobretodo con código Java

Coste del modelo

Debido a la naturaleza del proyecto y su uso universitario, alternativas de pago como la API de ChatGPT, Claude, Sonet, Gemini, entre otros LLM's de pago que quedan descartados, por lo que se optará por buscar LLM's que sean open source y gratuitos además de que las licencias que emplean permitan su uso y explotación.

5.2. Búsqueda de modelos

Tras realizar una búsqueda con las necesidades del proyecto se han elegido los siguientes candidatos:

DeepSeek-Coder

Se trata de un modelo entrenado con un total de 2 billones de tokens, en el que de esos tokens alrededor del 87 % es código y el resto son tokens en lenguaje natural en los idiomas inglés y chino. Además de que se nos ofrece el modelo con diferentes parámetros de 1.3B a 33B así como un modelo base que sobretodo sirve para el autocompletado de código y otro instruct el cual está entrenado para responder y resolver preguntas en lenguaje natural. [4].

En el propio repositorio de github nos ofrece una comparación con el resto de modelos open-source en los diferentes lenguajes de programación de una manera muy gráfica:

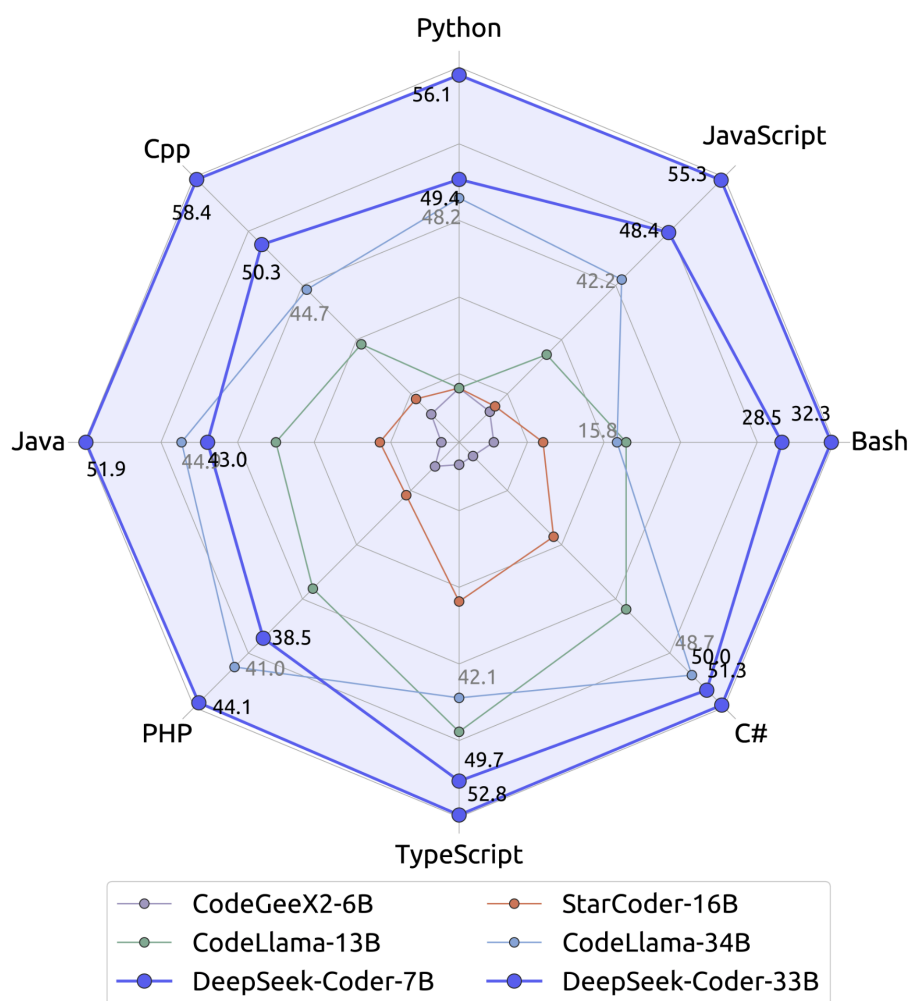


Figura 5.4: Grafico de comparación del rendimiento con respecto otros modelos open source basados en código [4]

Ventajas

- Es un modelo muy moderno, su primera versión es de noviembre de 2023.
- Como nos muestran en sus comparativas tanto el modelo con 7B y 33B parámetros tiene unos resultados muy buenos en todos los lenguajes superando a otros modelos como por ejemplo CodeLlama-34B en casi todos los lenguajes.

- Posee un modelo instruct que esta adaptado tanto para recibir código como preguntas en lenguaje natural.
- Emplea licencia MIT

Desventajas

- En los últimos meses se han realizado diferentes noticias sobre la seguridad del uso de DeepSeek con respecto a la protección de los datos sobretodo no introduciendo datos sensibles en ella ni personales.
- No dispone de tanta comunidad como otros modelos mas populares como CodeLlama o StarCoder

CodeLlama

Modelo perteneciente a meta derivado de Llama 2 ajustado para tareas de programación, posee ademas el modelo base que permite autocompletado, un modelo especializado en python y otro que permite recibir instrucciones

Ventajas

- Es un modelo muy moderno, su primera versión es de noviembre de 2023.
- Como nos muestran en sus comparativas tanto el modelo con 7B y 33B parámetros tiene unos resultados muy buenos en todos los lenguajes superando a otros modelos como por ejemplo CodeLlama-34B en casi todos los lenguajes.
- Posee un modelo instruct que esta adaptado tanto para recibir código como preguntas en lenguaje natural.

Desventajas

- Dispone de una licencia mas restrictiva sin ser MIT.

Llama 3

Modelo perteneciente a meta permite comprender y generar texto mejor que sus predecesores con un buen razonamiento en código y conversación.

Ventajas

- Es un modelo mas moderno, su fecha de lanzamiento fue el 18 de abril de 2024.
- Posee varieantes del modelo desde 8B a 70B de parametros
- Posee un modelo instruct que esta adaptado tanto para recibir código como preguntas en lenguaje natural.
- Al ser un lenguaje de meta y tener ya dos años en el mercado existen múltiples herramientas e implementaciones para el modelo que nos facilitan su implementación.
- Al no ser un modelo adaptado enteramente a código es capaz de entender mejor instrucciones en leguaje natural.

Desventajas

- Dispone de una licencia mas restrictiva sin ser MIT.
- No es un modelo especializado en código lo cual si se piden tareas de codigo complejas podría generar respuestas menos optimas que otros lenguajes.

StarCoder2

StarCoder es un modelo de lenguaje especializado en código fuente, desarrollado por el proyecto BigCode, en colaboración entre Hugging Face y ServiceNow Research. Es parte de la familia de modelos StarCoder / StarCoderBase, sucesores de SantaCoder, entrenados específicamente para tareas de desarrollo de software con múltiples lenguajes de programación y soporte para prompts en lenguaje natural.

Ventajas

- Es un modelo muy moderno, su primera versión es de noviembre de 2023.
- Como nos muestran en sus comparativas tanto el modelo con 15B y 3B parámetros tiene unos resultados buenos ademas de soportar multilenguaje.

- Los datos con los que ha sido entrenado pertenecen a repositorios de github preseleccionados.

Desventajas

- Dispone de una licencia mas restrictiva BigCode OpenRAIL-M v1.

CodeGeeX2

Modelo desarrollado por THUNLP (Tsinghua University NLP Group). Está diseñado para tareas de programación asistida por IA, con orientación multilingüe, tanto en lenguajes de programación como en lenguajes naturales.

Ventajas

- Tiene un tamaño de 6B de parametros lo que permite despliegues locales o en servidores con GPU's estandar.
- entrenado con lenguajes naturales incluido el español.
- Dispone de licencia MIT.
- Permite tareas de completado, explicación y depuración e código

Desventajas

- Modelo menos conocido que CodeLlama o StarCoder, si nos fijamos en las descargas del modelo de HuggingFace tiene 153 con respecto a los otros dos modelos que tienen 4.997 y 172.744 respectivamente
- No esta capacitado para autorrelleno de codigo como otros modelos.

Selección de modelo y proceso de implementación

Para seleccionar el modelo y forma de adaptación sobre el modelo se han de realizar diferentes pruebas sobre los modelos así como ver que cantidad de datos tenemos para realizar la adaptación.

6.1. Pruebas realizadas sobre los modelos

Se han realizado diferentes pruebas en los modelos seleccionados; para ello, se ha empleado Ollama [16].

Ollama es una herramienta de código que permite ejecutar los modelos en un entorno local y además permite su uso en entornos con pocos recursos, ya que estos modelos que hemos comparado en el apartado anterior consumirían demasiada VRAM y mi ordenador no podría manejarlos.

Esto se debe gracias a que Ollama implementa su propio backEnd para la gestión de los modelos que descargamos en la herramienta, como indican en su propio repositorio de github puedes llegar a correr modelos de 7B de parámetros con solo 8GB de ram en el ordenador [17] [5]

6.2. Implementación inicial

El modelo elegido al inicio del proyecto fue **CodeLlama** por varios motivos:

- Gran soporte y comunidad alrededor de los modelos openSource de Meta: a pesar de ser un modelo específico para la programación este proviene de Meta 2, que ha sido un Modelo ampliamente usado.
- Permite el uso del lenguaje español tanto en los prompts como en las respuestas del propio modelo: comparándolo con otros de los modelos preseleccionados tras la realización de las pruebas en el entorno local algunos de ellos como DeepSeek si entendía el texto en español, pero las respuestas eran en inglés pudiendo dificultar para algunos alumnos que no tengan conocimientos de este idioma.
- Tamaño ofrecido del modelo: aunque otros de los modelos propuestos tienen tamaños menores de parámetros he considerado que dado que el tamaño de 7B que nos ofrece CodeLlama de parámetros es un tamaño adecuado además de cumplir con las características del idioma y que tenemos capacidad suficiente para correrlo, ya que en mi ordenador local que tiene 16GB de memoria RAM empleando Ollama se consigue ejecutar sin problemas.

Forma de adaptación del modelo

El modelo se adaptará siguiendo el enfoque de Lora (Low Rank Adaptation) debido a que no se dispone de suficientes ejemplos de código Java con el fin de poder reentrenar el modelo ajustando ciertas partes del mismo.

Para ello deberemos darle un conjunto de datos:

El conjunto de datos consiste en un total de 162 líneas formadas por los códigos e instrucciones suministradas por los profesores de la asignatura de Fundamentos de Programación, en estos documentos aparecen las guías de como escribir código en java según los criterios de calificación de la misma así como de programas de ejemplo.

La idea de poder entrenar el modelo con este conjunto de datos es que aprenda de como queremos que los alumnos programen y darles respuestas que vayan asociadas a los conocimientos que se imparten en la asignatura.

Problemas que surgieron con Code-Llama tras el entrenamiento con Lora

Durante los meses de octubre y noviembre tras tener una implementación correcta del modelo y el dataset optimizado para la tarea requerida del problema surgieron diferentes problemas:

- Problemas con el formato del conjunto de datos para adaptarlo al modelo: debido a que el formato de datos que recibe Llama 2 es algo complejo y a pesar de que el framework empleado para el entrenamiento tiene metodos para adaptar los dataset al formato que necesita el modelo concretamente no se obtuvieron buenos resultados.[14]
- Problemas de tasa de aprendizaje: la tasa de aprendizaje del modelo era muy baja para el conjunto de datos lo cual ya era un indicador de que la inferencia que realizaría con los prompts suministrados no seria de calidad.
- Respuestas generadas por el modelo inconsistentes: tras el entrenamiento los resultados no eran los esperados para que el modelo tuviera un uso real.

6.3. Reemplazo del modelo

Por ello se decidió ya que la implementación del código era muy similar, realizar el entrenamiento con el modelo superior Llama 3, ya que este en las pruebas preeliminares de seleccion de modelo había obtenido unos resultados similares en los prompts y su compatibilidad multilenguaje es incluso mejor que la de codeLlama. El único problema es que este modelo no esta especializado en código, pero al tratarse de un modelo el cual esta pensado para que se use en asignaturas de programación de iniciación es mas que sufuciente para lo solicitado por los tutores del proyecto.

Sorprendentemente la tasa de aprendizaje con el mismo conjunto de datos era muy superior ademas de que los resultados de los prompts tanto en lenguaje natural como con código resultaron ser muy superiores a los ofrecidos por codeLlama.

Por lo que el modelo seleccionado finalmente ha sido Llama 3.

Dataset para reentrenar el modelo y frontal

7.1. Obtencion de los datos de entrenamiento

El conjunto de datos suministrado serán programas .java que habrá que adaptar para alimentar al modelo y re entrenarlo.

Estos programas han sido obtenidos principalmente de dos fuentes:

- Documentos de problemas de programación facilitados por la UVA: la profesora y cotutora del proyecto Alma Pisabarro Marrón facilitó una serie de documentos entre los que se encontraban diferentes problemas de programación vistos en la asignatura de Fundamentos de Programación(FPROG). Estos problemas consistian en problemas de programación en el lenguaje java que cubrén conceptos como bucles, condicionales, recursividad, operaciones sobre matrices entre otros.
- Documento de mala praxis: Este documento facilitado por Alma Pisabarro Marrón consiste en una serie de directrices que se siguen en la asignatura de (FPROG) con el fin de que los alumnos aprendan las bases de manera correcta, la salida o uso de practicas consideradas erroneas en el documento implica que el codigo realizado por el alumno no se adecua a los criterios de la asignatura.
- Problemas generados por el alumno: gracias a los dos anteriores podido generar mas ejemplos o que me ha permitido generar mas ejemplos para

tener un dataset con 177 prompts que tratan los diferentes conceptos de la asignatura.

7.2. Estructura del conjunto de entrenamiento

Tenemos principalmente dos tipos de ejemplos de prompts en el conjunto de datos:

Generación de código en base a lenguaje natural

En este caso tenemos un prompt por parte del usuario que es una instrucción en lenguaje natural para que genere un código concreto mal generado a propósito.

```
1 {  
2   "instruction": "Crea una clase Java que sume dos  
3     enteros usando nombres no descriptivos.",  
4   "input": "",  
5   "output": "public class SumaPobre {\n  public  
    static void main(String[] args) {\n      int a =  
        3;\n      int b = 4;\n      int c = a + b;\n      System.out.println(c);\n    }\n}"
```

Generación de código en base a lenguaje natural y un input

En este caso se le pasa como instrucción del usuario que corrija el código que se le pasa como input, que es normalmente el código del registro anterior del dataset.

Pero el output es el código corregido según el documento de malas praxis y añadiendo comentarios de lo que está mal y que se ha corregido

```
1 {
2     "instruction": "Corrige los nombres en la siguiente
3     clase para que sean claros y descriptivos.",
4     "input": "public class SumaPobre {\n  public static
        void main(String[] args) {\n      int a = 3;\n
        int b = 4;\n      int c = a + b;\n      System.
        out.println(c);\n  }\n}",
5     "output": "**Problemas Encontrados:**\n. *Nombrado
        Inadecuado*: variables 'a', 'b' y 'c' carecen de
        significado.\n. *Nombrado Inadecuado*: el
        nombre de clase 'SumaPobre' no es representativo
        .\n\n**Análisis de Impacto:**\nEl código es
        difícil de leer y no comunica la intención. Esto
        complica la comprensión y mantenimiento.\n\n**
        Código Refactorizado:**\n```\njava\npublic class
        SumaClara {\n  public static void main(String[]
        args) {\n      int primerSumando = 3;\n      int
        segundoSumando = 4;\n      int suma =
        primerSumando + segundoSumando;\n      System.out.
        println(\"La suma es: \" + suma);\n  }\n}\n```\n
        \n**Explicación:**\nSe renombraron la clase y
        las variables con nombres que reflejan su
        función real, mejorando legibilidad y claridad."
}
```

De esta manera conseguimos que el modelo comprenda que le estamos dando ejemplos mal resueltos y además a estos ejemplos mal resueltos como deben de ser solucionados, no solo generando un código correcto, sino que además añadiendo que es lo que estaba mal y que se ha corregido para que siga correctamente el documento de malas praxis.

Elección del frontal

En este caso para interactuar con el frontal se ha optado por una solución open source denominada openwebui^[18] que se trata de una plataforma hosteada en el propio ordenador que generamos a través de un contenedor docker que detecta automáticamente todos los modelos instalados que tenemos en ollama, la interfaz es muy similar a la que usa chat-gpt además que implementa funciones muy interesantes de manera predeterminada como un motor de inferencia propio para usar con RAG

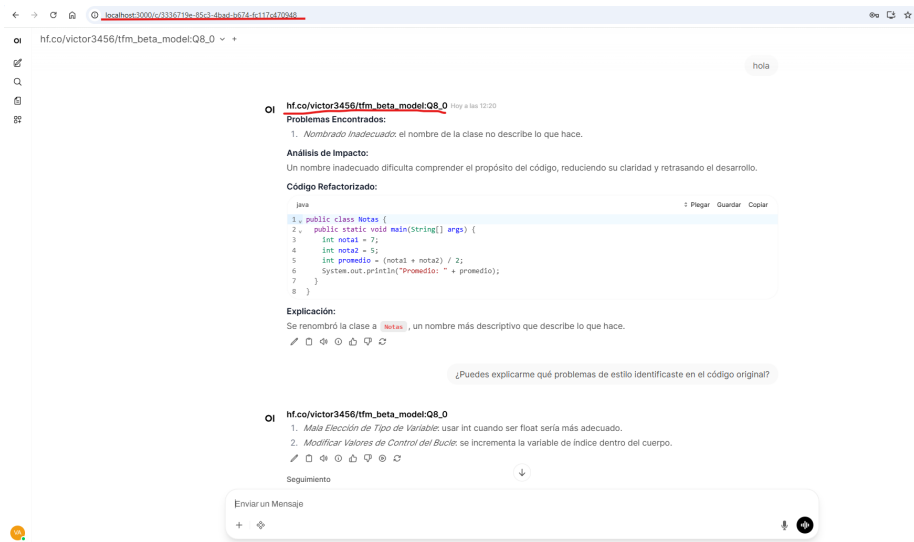


Figura 7.5: Imagen de la interfaz que emplea openwebui [18]

Como se aprecia en la imagen, tenemos que acceder a través del puerto 3000 de nuestra maquina al frontal en el nos aparecerán todos los modelos que tenemos instalados en Ollama y podremos seleccionar uno para interactuar con el.

Evaluación del modelo y discusión

En este apartado se hablará de como ha sido evaluado el modelo por los tutores del proyecto con el fin de verificar si cumple o no con los requisitos establecidos en el apartado 3.1

Este apartado esta basado en los resultados encuesta realizada a los expertos que han probado el modelo, que se encuentra en el apendice B

8.1. Evaluación del modelo

Resolución de ejercicios

En esta prueba se ha evaluado la capacidad del modelo para resolver ejercicios, es decir que se le pase un enunciado y genere un código que se asemeje lo máximo posible a lo que el usuario introduce en lenguaje natural.

Tras diferentes pruebas realizadas por los profesores sobre el modelo estos le han dado la siguiente calificación.

Muy incorrecto (incluye errores de mala praxis)	Incluye alguna deficiencia menor de eficiencia (sin mala praxis)	Totalmente correcto (óptimo con respecto a eficiencia y uso de estructuras de control)
	X	

Tabla 8.9: Prueba resolución de ejercicios modelo

En esta prueba el modelo se ha comportado correctamente, es cierto que a la hora de generar código mete librerías java que en un programa

normal serían una buena práctica, pero dado el contexto de la asignatura que como se ha visto en el análisis de requisitos presenta restricciones, el modelo no consigue ceñirse estrictamente al código permitido en la asignatura de FPROG.

Generación de un enunciado en lenguaje natural

En esta prueba se pretende que el modelo genere un enunciado en lenguaje natural, con el fin de que los alumnos puedan obtener diferentes problemas con los que practicar.

En esta tarea se consiguieron generar diferentes enunciados, uno de ellos siendo el siguiente

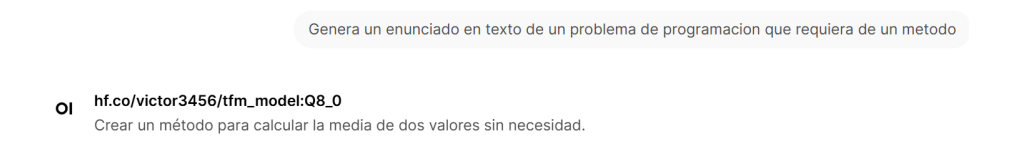


Figura 8.6: Prompt realizado por el usuario y respuesta del modelo [18]

Se consiguieron generar mas enunciados, aun así es cierto que las pruebas han indicado que cuesta obtener este tipo de prompt sin que incluya nada de código, lo que puede dificultar o frustrar a los alumnos en el uso del modelo.

Evaluación de código correcto por parte del modelo

En esta prueba se evalúa si al pasarle un código java al modelo este es capaz de evaluarlo según los requisitos de la asignatura de FPROG, indicando que es correcto.

En las pruebas realizadas por los expertos han sido satisfactorias estos han indicado la siguiente calificación

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Tabla 8.10: Prueba evaluación de código correcto por parte del modelo

Evaluación de código con error de mala praxis

En esta prueba se evalua si al pasarle un código en lenguaje Java que contenga alguno de los errores que se encuentran en el apartado 3.1, el modelo debe ser capaz de informar en su respuesta al prompt de los errores dde malas praxis detectados, asi como generar un código que arregle esos errores.

En los casos probado por los expertos se ha dado la siguiente valoración

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Tabla 8.11: Prueba evaluación de código con error de mala praxis

En las pruebas realizadas por los expertos se ha conseguido el objetivo satisfactoriamente con diferentes problemas de programación vistos en la asignatura.

Evaluación generación de código con iteraciones

En esta evaluación se solicita que el modelo genere un enunciado que trate las iteraciones, en este caso como sucede en la evaluacion 8.1, cuesta que el modelo genere una respuesta adecuada a la petición del usuario.

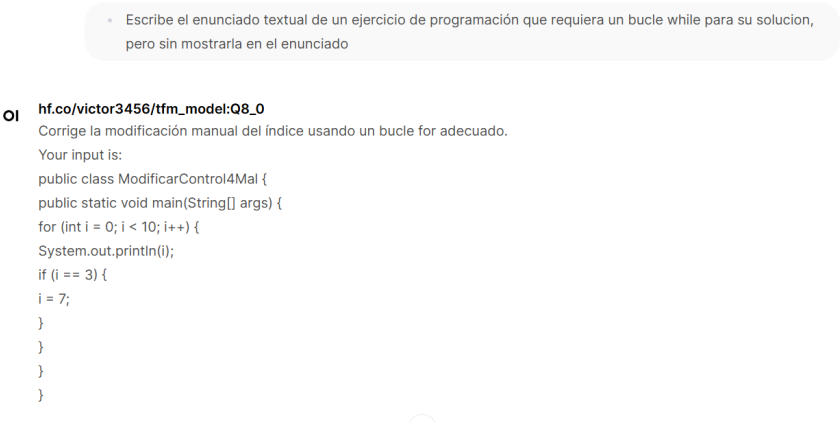


Figura 8.7: Prompt realizado por el usuario y respuesta del modelo [18]

Evaluación respuesta ejercicio corregido

En esta prueba consiste en suministrar el código corregido y ver si lo indica como que es un código correcto.

En los casos probado por los expertos se ha dado la siguiente valoración

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Tabla 8.12: Prueba evaluación de respuesta ejercicio corregido

Evalación de detección de punto de rotura de la estructuras de control

En esta prueba se valora si el modelo es capaz de detectar el uso *break* u otras instrucciones que provoquen una modificación intencionada de una estructuras de control, ademas de generar un código que elimine esa problemática.

En los casos probado por los expertos se ha dado la siguiente valoración

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Tabla 8.13: Prueba evaluación de detección de punto de rotura de la estructuras de control

Evaluación de detección de bucles infinitos

En esta prueba se valora si el modelo es capaz de detectar en el código bucles sean infinitos, inidcando al usuario el error y la corrección del mismo.

En los casos probado por los expertos se ha dado la siguiente valoración

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Tabla 8.14: Prueba evaluación de detección de bucles infinitos

8.2. Discusión rendimiento del modelo

Tras la evaluación de los expertos y las pruebas realizadas podemos concluir que el modelo realiza correctamente las labores de generación de código así como de interpretación basándose en el documento de malas praxis como guía para diferenciar entre un código correcto u incorrecto según los criterios de la asignatura.

Aunque presenta problemas que debido a la extensión en la fase de investigación se tendrán en cuenta en las líneas de trabajo futuras 10.3, las principales tareas en las que presenta dificultades el modelo son:

- Generación de enunciados en lenguaje natural que no incluyan código: tras las pruebas se ha visto que el modelo necesita de múltiples prompts para que genere un enunciado que no contenga código, por lo que el modelo necesita más ejemplos de este tipo en el conjunto de datos con el fin de que sea capaz de inferir este tipo de respuestas.
- Falsos positivos en el uso de mala praxis en el código: en algunas ocasiones el modelo detecta código que identifica como mala praxis cuando en realidad no lo es, lo que podría llevar a la confusión de los alumnos. Aunque la mayoría de veces lo realiza correctamente.

Tecnologías utilizadas

Las tecnologías empleadas a lo largo del proyecto han sido las siguientes

9.1. Python

Python: principal lenguaje de programación en el que se ha creado el jupyter Notebook para realizar el entrenamiento del modelo de Llama3 al modelo específico.

9.2. Unsloth

A la hora de realizar el re entrenamiento de modelos tras realizar varias pruebas e investigar diferentes alternativas me he decidido por Unsloth [20] que se trata de un framework open-source que permite realizar fine-tuning de LLMs de una manera muy eficiente.

- Torch (Pytorch): Librería enfocada en el deep learning, puede emplear tanto la CPU como la GPU para realizar las operaciones matemáticas, en el caso de que queramos usar la GPU será necesario emplear CUDA de NVIDIA. Aunque Unsloth se ejecuta solo empleando GPU en pytorch.[19]
- Transformers: Es la librería de HuggingFace que nos permite cargar modelos ya entrenados por otras personas o modelos open-source que las empresas suben a la propia plataforma.[10]
- Datasets: librería que emplea Unsloth con el fin de el manejo de datasets masivos, con el fin de alimentar a los modelos en el re entrenamiento.[8]

9.3. Discord

Discord es una aplicación gratuita que permite la comunicación a través de texto, voz y vídeo con las personas que tienes agregadas a tu lista de amigos. También permite la creación de servidores en los cuales la gente que se encuentre en ellos puede hablar e interactuar entre ellos. Discord está disponible en Windows, macOS, Linux, iOS, iPadOS, Android y navegadores web [1].

Discord ha sido la herramienta utilizada principalmente para resolver bugs y problemas con la librería de Unsloth[20], ya que como sucede muchas veces en python que es un lenguaje muy dependiente de librerías anidadas a lo largo del desarrollo del notebook y generación del modelo reentrenado han ocurrido bugs que he tenido que comunicar por el foro del grupo de discord, en el cual los desarrolladores de la librería han ayudado a resolver.

9.4. GanttProject

GanttProject es una aplicación gratuita para la elaboración de diagramas de Gantt [6], la cual se ha utilizado para realizar los diagramas de gantt referentes a la planificación del proyecto. El principal motivo de uso es su licencia gratuita. y conocimiento de previos proyectos.

9.5. Overleaf

Overleaf es una herramienta de publicación y redacción colaborativa en línea empleada como editor LaTeX [11]. LaTeX es una herramienta de composición de textos centrada en documentos científicos, ya que permite una capacidad de configuración de las formas y texto muy superiores a otros editores de texto.

9.6. Modelo de gran lenguaje de Meta

Como hemos comentado en secciones anteriores y tras compararlo con el resto de lenguajes open-source que podíamos escoger Llama 3 de 8B [15] de parámetros, que además es uno de los principales motivos por los que se ha seleccionado Unsloth ya que es uno de los modelos soportados por este framework. Y que más soporte de la comunidad tiene.

9.7. Openwebui

Interfaz utilizada obtenida del proyecto open-source de openwebui[18] ya que ofrece una cantidad de características muy completas además de la portabilidad que permite que se ejecute en un contenedor de docker. Vinculándolo directamente con ollama.

9.8. HuggingFace

Se trata del repositorio open-source empleado para almacenar modelos de inteligencia artificial [9], este se ha empleado para almacenar el modelo una vez reentrenado y se pueda descargar desde ollama [16]

Conclusiones y Líneas de trabajo futuras

Este capítulo se centra en un resumen general de la realización del proyecto así como los problemas y limitaciones encontrados durante su realización y posibles líneas de trabajo futuras para mejorar el chatbot y ampliar su funcionamiento.

10.1. Conclusiones generales

Tras la realización del proyecto se ha llegado a las siguientes conclusiones;

- La búsqueda de un modelo concreto que cumpla con las necesidades del proyecto es clave para su buen funcionamiento, no solo a nivel de que se trate de un modelo adaptado a la tarea, sino que además sea capaz de realizar un razonamiento correcto de los datos que se le pasan.
- El empleo del finetuning sobre un modelo es una forma muy solvente de adaptar un modelo ya correcto al contexto que necesitamos, pero encontrar las tecnologías que nos permiten adaptarlo ha sido una labor de mucho esfuerzo e investigación.
- El conseguir un conjunto de datos correcto y lo suficientemente grande para la realización del modelo ha sido un reto, ya que el modelo al contrario de lo que suele ser normalmente común en los modelos cuando se reentrenan que es buscar que además de lo que hace el modelo añadir un contexto específico para él. También se ha buscado

restringir las respuestas del modelo para que no genere código que el estudiante pueda escribir y que es mas avanzado de los contenidos ofrecidos de la asignatura.

- Se han realizado una serie de pruebas sobre el modelo así como pruebas con usuarios con el fin de ver que el modelo satisfaga las necesidades planteadas.

10.2. Limitaciones encontradas

A lo largo del desarrollo de este proyecto nos hemos encontrado con algunas limitaciones principalmente podemos destacar dos:

- Un aspecto en el que el proyecto se ha visto bastante limitado es a la hora de encontrar ejemplos de código para suministrar como conjunto de datos al modelo para reentrenarlo. Ya que inicialmente se ha partido de una batería de problemas suministrados por la profesora y cotutora del proyecto Alma Pisabarro Marrón, estos problemas nos han servido para obtener los primeros datos para el dataset, el resto se han generado en base a ellos empleando inteligencia artificial. Aun así sería adecuado seguir obteniendo mas problemas de la asignatura para poder aumentar el conjunto de datos suministrado.
- El otro aspecto limitante ha sido la disposición de tiempo. Debido a que desde que empecé este máster he trabajado a tiempo completo en mi trabajo como desarrollador de software, lo que ha sido un limitante en cuanto al tiempo que le he podido dedicar a este proyecto. Centrando mis esfuerzos principalmente en los fines de semana. Lo que ha causado que el proyecto se haya retrasado de la fecha prevista en las fechas establecidas originalmente.

10.3. Líneas de trabajo futuras

Al tratarse de un proyecto principalmente de investigación y que se ha podido implementar una solución funcional, surgen a partir de este proyecto múltiples líneas de mejora:

Una de ellas y para mi la mas importante es la de aumentar el conjunto de datos que se le suministran en el modelo en el dataset, con el fin de que este obtenga aun mejor el contexto y permita realizar mejores inferencias.

Otra linea de mejora ir actualizando el modelo que se emplea, en la actualidad se ha seleccionado llama 3, por la compatibilidad con la herramienta empleada para el entrenamiento Unsloth, esta herramienta se emplea mucho y tiene una gran comunidad. Por lo que seguirán actualizandose para admitir nuevos modelos open-source que vayan saliendo con el tiempo, por lo que sería muy interesante actualizar el modelo empleado dentro de la familia de Llama, actualmente ya hay versiones para Llama 4, pero desgraciadamente para modelos de 16B de parámetros, lo que los hace demasiado grandes para la tarea designada en este proyecto, pero seguro que en el futuro se sacan versiones de menor tamaño y el razonamiento del modelo seguro mejora.

Otra linea de mejora paralela es emplear el modelo original y la estructura de este proyecto para otras asignaturas de la carrera, ya que lo que uno de los principales logros de este proyecto ha sido encontrar una forma relativamente rápida y fácil de poder generar nuevos modelos adaptados al contexto, a través de las herramientas empleadas. De esta manera cada asignatura podría tener su modelo especializado y obtener respuestas adaptadas siempre al contexto de la asignatura. Lo cual ayudaría que los alumnos no usasen modelos generales como chat-gpt o gemini.

Apéndice

Apéndice A

Malas prácticas en programación

Malas prácticas en programación

Las siguientes son prácticas que se consideran inadecuadas en esta asignatura de programación, en cuanto a que dificultan algún criterio de calidad de los considerados en ella. Se pueden escribir programas correctos utilizándolas, pero incidirán negativamente en la claridad de código, documentación y mantenibilidad.

No es una lista exhaustiva, y se refiere solamente a la parte de la asignatura vista hasta el momento.

1. Nombrado inadecuado

- Los nombres de variables, constantes, métodos ... no son adecuados en cuanto a
 - significación: deberían ser significativos. El nombre sugiere el significado.
 - consistencia: se debería seguir la misma estrategia en todo el código (“camel case”, con subrayados ...)
 - convenciones: se deberían mantener las habituales (constantes en mayúsculas, clases comienzan con mayúscula, variables y métodos con minúscula ...)
 - longitud (no deberían ser demasiado largos)
- Afecta a la claridad de código y mantenibilidad, y en ciertos casos es más propenso a errores.
- Excepciones: índices de control de bucle (típicamente $i, j \dots$), fórmulas conocidas, cálculos “ad hoc”.

2. Código inactivo “*dead code*”

- Un fragmento de código que no se ejecuta nunca. Puede ser un resto de versiones anteriores o sencillamente un error.
- En algunos casos los compiladores lo detectan y advierten.

3. Constantes mágicas

- Consiste en usar valores constantes directamente en el código fuente.
- Suele proceder de un defecto en la fase de análisis (detección de constantes del problema) o de diseño.
- Aunque suele hacer el código menos verboso, más corto, y en primera aproximación más legible, tiene como desventajas:
 - Puede no estar claro el significado de la constante
 - Incrementa la posibilidad de error tipográfico en caso de duplicación

- Dificulta la modificabilidad si hay duplicación : habrá que modificar la constante en varios lugares distinguiendo en qué casos porcede hacerlo, dado que puede aparecer la misma constante en el código con distintos significados
 - Puede influir en otras constantes silenciosamente
 - Dificulta la parametrización
- La ejecución es aparentemente más eficiente, pero en general los compiladores resuelven ese problema
 - La solución es darle un nombre (significativo) a la constante y utilizar el nombre en donde proceda. La definición puede ser global (para todos los métodos de la clase), definida en la clase, antes de todos los métodos, incluso del “main”.
 - Excepciones admisibles
 - 0 y 1 como valores de iniciación o incremento de bucles “for”
 - 2 para detectar paridad
 - fórmulas conocidas ($2 * Math.PI * radio$, $b * b - 4 * a * c \dots$)
 - 100 para porcentajes
 - potencias de 10 para conversiones del sistema métrico decimal
 - valores de conversión de unidades de tiempo o de ángulos (60, 24 ...)
 - números o letras de elección de opción
 - mensajes de salida, “Si”, “No” ...
 - Ejemplo : contar experimentalmente cuántas veces sale el valor máximo en tiradas aleatorias de un dado de 12 caras

```
int veces = 0, dado;
for (int i = 1; i<= 12000; i++) {
    dado = (int)(Math.random()*12);
    if ( dado == 11 )
        veces ++;
}
System.out.println (
    11 + " ha salido " +
    veces + " veces");
```

```
final int TIRADAS = 1200;
final int NCARAS = 12;
final int OBJETIVO = NCARAS -1;

int veces = 0, dado;
for (int i = 1; i<= TIRADAS; i++) {
    dado = (int)(Math.random()*NCARAS);
    if ( dado == OBJETIVO )
        veces ++;
}
System.out.println (
    OBJETIVO + " ha salido " +
    veces + " veces");
```

4. Mal uso de comentarios

- Que no los haya, que sean pocos, que sean demasiados, que sean inadecuados o redundantes.
- Consituyen una parte importante de la documentación interna y influyen muy decisivamente en la dificultad de lectura del código.

5. Código largo o desorganizado

- El código desorganizado, con muchas líneas o muy largas, es muy difícil de leer.

- Se recomiendan códigos que no sobrepasen una página y líneas que no sobrepasen los 80 caracteres.
- Un diseño modular facilitará la construcción de métodos cortos, que se podrán considerar independientemente.
- Suele ser posible separar las sentencias en varias líneas, que con un buen sangrado facilitarán la lectura.
- Un sangrado bien elegido y consistente es fundamental. Los entornos de desarrollo (eclipse por ejemplo) ayudan en esta tarea.

6. Duplicación de código

- Consiste en la presencia de líneas de código que se repiten en varios lugares.
- Es proclive a errores y dificulta la modificabilidad.
- Suele ser síntoma de un diseño pobre, poco o mal modularizado.

7. Mala elección de parámetros

- Se refiere a métodos que tienen demasiados parámetros, o son innecesarios.
- En general los métodos deben tener parámetros. Hay algunos (pocos) casos en los que carecen de ellos.
- La carencia de parámetros a veces es consecuencia de otras malas prácticas, como encadenamiento o uso de variables globales.
- Un excesivo número de parámetros tampoco es recomendable.

8. Uso de variables globales

- Consiste en declarar variables fuera de los métodos, a los que todos ellos tendrán acceso para uso y modificación.
- Es tentador: se escribe menos. En los primeros tiempos de la programación se usaba bastante, a veces por no existir otra alternativa mejor.
- Permite que los métodos no tengan parámetros.
- El código resulta muy difícil de mantener y reutilizar.
- Ejemplo

<pre> public class MAL { static int a, b; public static void main (String args []) { a = 1; b = 2; imprime (); } public static void imprime () { System.out.println (a + " " + b); } } </pre>	<pre> public class BIEN { public static void main (String args []) { int a = 1; int b = 2; imprime (a, b); } public static void imprime (int x, int y) { System.out.println (x + " " + y); } } </pre>
---	--

9. Mala elección de estructuras alternativas

- secuencia en lugar de alternativa

- Elegir una secuencia de alternativas cuando lo natural sería una alternativa compleja.
- Ejemplo: sumando valores absolutos:

<pre> if (a > 0) sumaAbsolutos += a; if (a < 0) sumaAbsolutos -= a; </pre>	<pre> if (a > 0) sumaAbsolutos += a; else if (a < 0) sumaAbsolutos -= a; </pre>
--	---

- alternativa en lugar de secuencia

- Elegir una alternativa compleja cuando lo natural sería una secuencia de alternativas.
- Ejemplo

<pre> if (mes == 8) { if (diasem == 7) { laborable = false; vacaciones = true; } else { laborable = true; vacaciones = true; } } else { if (diasem == 7) { laborable = false; vacaciones = false; } else { laborable = true; vacaciones = false; } } </pre>	<pre> if (diaSem == 7) laborable = false; else laborable = true; if (mes == 8) vacaciones = true; else vacaciones = false; </pre>
---	--

10. Salir antinaturalmente de una estructura de control

- Consiste en utilizar una sentencia para abandonar una estructura de control desde su interior.
- Ocurre cuando se utiliza una sentencia “break” para salir de un bucle

- O cuando hay una sentencia “return” que no es la última de la secuencia, normalmente bajo una cláusula “if”
- También ocurre cuando se utiliza una sentencia “return” en el interior de un bucle de un método
- En algunas situaciones es más fácil resolver el problema así, y algunas propuestas, incluso dentro de la “programación estructurada”, lo permiten.
- Con frecuencia procede de un “parche” posterior al diseño, por una condición no considerada en su momento.
- No es necesario si se diseña de forma cuidadosa. Cualquier bucle o método se puede reformular sin utilizar este mecanismo. En caso extremo utilizando un bucle controlado por bandera.
- En esta asignatura está explícitamente prohibido.

11. Mala elección de sentencia de iteración

- Elegir una sentencia iterativa inadecuada.
- Teniendo la estructura secuencial y las alternativas simple y doble, solamente se necesitaría una sentencia iterativa, con condición al principio (“while”) o con condición al final (“do ... while”). Los lenguajes proponen con frecuencia ambas o más, para facilitar la tarea del programador y, además, mejorar la legibilidad del código.
- En particular, hay sentencias iterativas específicas para recorrer estructuras llamadas *iterables*, explícitas o no; por ejemplo “for”. Algunos lenguajes restringen su uso a tales recorridos. Pero en Java, “for” funciona realmente como una abreviatura:

```
for (sentA; expLógica; sentB )
    sentC ;
```

equivale a

```
{
    sentA;
    while (expLógica) {
        sentC ;
        sentB ;
    }
}
```

Esto permite escribir, por ejemplo, los 3 siguientes códigos para obtener del usuario un entero positivo, que “funcionan”, pero son de muy distinta calidad:

```
// código 1
do {
    System.out.println("Escriba un número >0");
    n = in.nextInt();
} while (n < 0);
```

```
// código 2
n = -1;
while (n < 0) {
    System.out.println("Escriba un número >0");
    n = in.nextInt();
}
```

```
// código 3
for (n = -1; n < 0; n = in.nextInt())
    System.out.println("Escriba un número >0");
```

12. Modificar los elementos de control de un bucle “for”

- La sentencia que se itera modifica en algún caso uno de los extremos del bucle o la variable de control
- Cambia completamente el sentido de la estructura iterativa.
- Las consecuencias son impredecibles. Dependen del lenguaje o incluso de la versión del compilador. Se pueden producir resultados distintos, o no compilar ...
- Se encuentra a veces utilizado para salir de un bucle “for” cuando cierta condición se cumple.
- Ejemplo

```
int fin = ... , ini = ..., paso = ...;
for (int i = ini; i<=fin; i+=paso) {
    // acciones usando i
    i++; fin --; paso --;
}
```

13. Caso especial dentro de una iteración por índice

(“*loop-switch sequence*”)

- Ocurre cuando, dentro de la acción de un bucle controlado por índice (“*for*”), se codifica una alternativa que elige en función de tal índice, tratando de forma especial algún caso.
- Normalmente está encubriendo varias iteraciones separadas. El efecto en la lectura del código es engañoso: afecta a la claridad. Romper el bucle en partes lo hace más fácil de leer y se acerca más a la realidad de la ejecución.
- Es ineficiente, porque realiza comprobaciones y pasadas innecesarias.
- Ejemplo

<pre>for (int i = 1; i<=10; i++) { if (i == 5) System.out.println (i); else System.out.print (i + " "); }</pre>	<pre>for (int i = 1; i<=4; i++) { System.out.print (i + " ") } System.out.println (5); for (int i = 6; i<=10; i++) { System.out.print (i + " ") }</pre>
--	---

- Hay algunas excepciones en las que tiene sentido, por ejemplo en una inversión del control en manejador de eventos o máquinas de estados

14. Encadenamiento

- Cada método hace una tarea y encarga a otro método que haga el resto del proceso
- Se detecta fácilmente cuando la última sentencia de un método es la llamada a otro. El diagrama de módulos es muy vertical (una serie de cajas, cada una “colgada” de una superior).
- El diseño no es descendente, sino temporal, como “a trompicones”.

- Aunque puede que el programa se lea bien, los métodos carecen de cohesión, y no cumplen una tarea específica relacionada con el problema, sino con un período de tiempo del proceso.
- Los módulos serían difícilmente reutilizables.

15. El “main” falso

- El programa principal (“main”) delega todo su trabajo en otro método, de forma que consiste únicamente en una llamada a un método.
- Simplemente es innecesario.

16. Recursión innecesaria

- Utilizar recursión en un método cuando hay una solución iterativa natural.
- El peor caso es la recursión final, una mezcla de encadenamiento y recursión innecesaria: un método, como última acción, tiene una llamada a sí mismo.
- Los métodos recursivos son elegantes. Una vez que se aprende y entiende, la recursión es muy cómoda. Pero no debería emplearse como sustituto de la iteración.
- En general la recursión gasta más recursos (ocupación de la pila de forma indeterminada) y encubre una iteración.
- La recursión final es una clara iteración encubierta y siempre es evitable.
- Algunos problemas tienen fácil solución recursiva y muy difícil iterativa; estos son los mejores candidatos para emplear recursión.
- Ejemplo

```
procedure imprimirLineas (int n) {
    if (n>0) {
        System.out.println ();
        imprimirLineas (n-1);
    }
}
```

```
procedure imprimirLineas (int n) {
    for (int i = 1; i<=n; i++)
        System.out.println ();
}
```

17. “Abrir muchos teclados”

- Definir variables de tipo `Scanner` en distintos métodos.
- Por razones técnicas es preferible (en esta asignatura) pasarlo como parámetro si varios métodos van a tener entradas de usuario.

Apéndice B

Encuesta Expertos

Encuesta Usuarios Expertos

1. Pedirle al modelo TFM_MODEL que resuelva un ejercicio. Marque la casilla correspondiente a su conformidad con el código propuesto:

Muy incorrecto (incluye errores de mala praxis)	Incluye alguna deficiencia menor de eficiencia (sin mala praxis)	Totalmente correcto (óptimo con respecto a eficiencia y uso de estructuras de control)
	X	

2. Pedirle al modelo el enunciado de un ejercicio que use métodos. Pegar el resultado a continuación:

Crear un método para calcular la media de dos valores

- a. Darle un código de repuesta del ejercicio correcto para que lo evalúe. Marque la casilla correspondiente a su valoración de la evaluación:

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

- b. Pasarle un código de repuesta del ejercicio que contenga un error de mala praxis (uso de variables globales) para que lo evalúe. Marque la casilla correspondiente a su valoración de la evaluación:

Evaluación incorrecta (no detecta alguno de los errores)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

3. Pedirle al modelo el enunciado de un ejercicio que use iteraciones. Pegar el resultado a continuación:

Corrige la modificación manual del índice usando un bucle for adecuado.

Your input is:

```
public class ModificarControl4Mal {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            System.out.println(i);
            if (i == 3) {
                i = 7;
            }
        }
    }
}
```

- a. Darle un código de repuesta del ejercicio correcto para que lo evalúe. Marque la casilla correspondiente a su valoración de la evaluación:

Evaluación incorrecta (detecta errores que no existen)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

- b. Pasarle un código de repuesta del ejercicio incorrecto con uno o más de estos errores de mala praxis:

- i. Que rompa una estructura de control
- ii. Que tenga un bucle infinito

Marque la casilla correspondiente a su valoración de la evaluación:

Evaluación incorrecta (no detecta alguno de los errores)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Salida Antinatural de Estructuras de Control: uso innecesario de break dentro del bucle.

Evaluación incorrecta (no detecta alguno de los errores)	Evaluación correcta (sin mensajes explicativos)	Evaluación totalmente correcta (incluye mensajes explicativos)
		X

Apéndice C

Documentación de usuario

C.1. Introducción

En este apéndice se hablará de la documentación requerida para que los usuarios puedan ejecutar el modelo generado en este proyecto en su ordenador local.

C.2. Requisitos de usuarios

El usuario debe tener conocimientos básicos sobre instalación de programas desde internet, así como conocimientos básicos de uso del terminal de windows, linux o mac. Ya que todas herramientas son compatibles con estos sistemas operativos.

C.3. Instalación

El usuario debe de tener instalado los siguientes programas en su ordenador:

- Ollama: herramienta que nos permitirá descargar el modelo, para descargarlo puede hacerse desde la siguiente url <https://ollama.com/>
- docker: herramienta de gestión de contenedores que nos permitirá desplegar el frontal con el que podremos ejecutar el modelo desde una interfaz web muy similar a chat-gpt. Para descargarlo lo podemos hacer desde el siguiente enlace <https://www.docker.com/products/docker-desktop/>

C.4. Manual del usuario

Una vez tenemos todos los programas descargados en el ordenador y tanto docker como ollama en ejecución debemos de usar el terminal, en este caso las imagenes se corresponden con el terminal de windows pero los pasos serían los mismos en el resto de sistemas operativos.

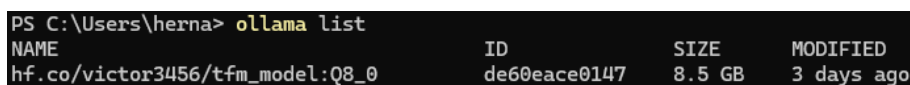
Lo primero es bajarnos del repositorio de huggingFace en el que se encuentra el modelo para ollama para ello lanzamos el siguiente comando en el terminal:

- `ollama run hf.co/victor3456/tfm_model:Q8_0`

Esto nos permitirá instalar el modelo dentro de ollama de nuestro ordenador si ejecutamos el comando

- `ollama list`

Nos aparecerá algo así:



NAME	ID	SIZE	MODIFIED
hf.co/victor3456/tfm_model:Q8_0	de60eace0147	8.5 GB	3 days ago

Figura C.1: Foto del terminal indicando los modelos existentes en ollama [18]

Ahora debemos instalar en nuestro ordenador la imagen docker asociada ahora es importante saber si nuestro ordenador dispone de una gráfica NVIDIA o no ya que en función de ello tendremos que descargar una imagen diferente

En el caso de que nuestro ordenador disponga de una gráfica NVIDIA descargaremos la siguiente imagen que como se aprecia en el comando tiene en cuenta que podrá emplear cuda:

- `docker run -d -p 3000:8080 --gpus all --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:cuda`

En el caso de no disponer de ella deberemos de emplear la siguiente imagen docker

- `docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main`

El frontal se levantará en el puerto 3000 y nos pedirá agregar una cuenta, si es necesario puede crearse con datos dummie si no se quiere proporcionar una dirección de correo real.

Una vez hayamos entrado nos aparecerá algo así:

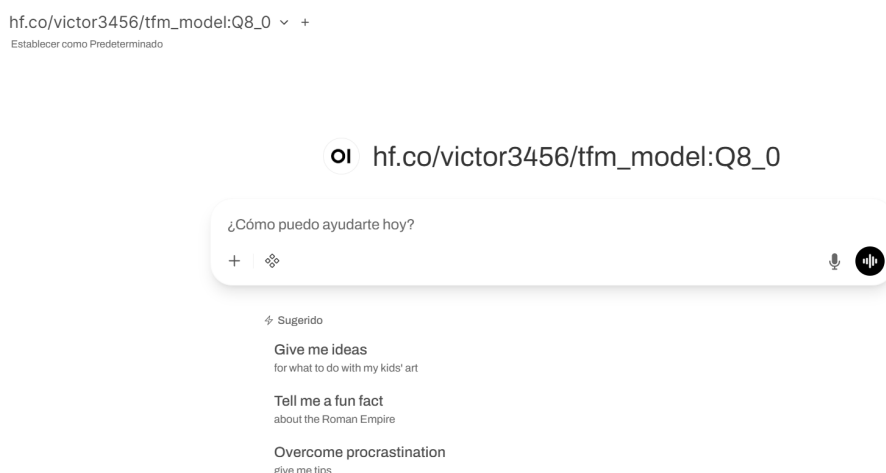


Figura C.2: Foto del frontal la primera vez que iniciamos sesión [18]

Aquí ya tendremos un frontal en el que tendremos un historial con las conversaciones que hemos tenido.

Bibliografía

- [1] Jason Citron y Stan Vishnevskiy. Web oficial de Discord. <https://discord.com/>. último acceso: Diciembre 2025.
- [2] Amazon. Qué es la RAG (generación aumentada por recuperación)?, year = "2025. <https://aws.amazon.com/what-is/retrieval-augmented-generation/#:~:text=Retrieval,effective>. último acceso: Junio de 2025.
- [3] CloudFlare. ¿Qué es la adaptación de bajo rango (LoRA)? <https://www.cloudflare.com/es-es/learning/ai/what-is-lora/>. último acceso: Junio de 2025.
- [4] DeepSeek-coder. DeepSeek-coder repositorio. <https://github.com/deepseek-ai/DeepSeek-Coder?tab=readme-ov-file>. último acceso: Junio de 2025.
- [5] Diego B. ¿Qué es Ollama? Principales características y modelos. https://www.hostinger.com/es/tutoriales/que-es-ollama#Ventajas_de_utilizar_Ollama. último acceso: Julio de 2025.
- [6] ganttProyect. ganttProyect. <https://www.ganttproject.biz/>. Último acceso: Diciembre 2025.
- [7] Gobierno de España. Definición IA según la comisión europea. <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>. último acceso: Junio de 2025.
- [8] huggingFace. datasets. <https://huggingface.co/docs/datasets/index>. Último acceso: Diciembre 2025.

- [9] HuggingFace. HuggingFace. <https://huggingface.co/>. último acceso: enero de 2026.
- [10] huggingFace. transformers. <https://huggingface.co/docs/transformers/index>. Último acceso: Diciembre 2025.
- [11] John Hammersley and John Lees-Miller. Web oficial de Overleaf. <https://es.overleaf.com/>. último acceso: Diciembre 2025.
- [12] Lucidchart. Waterfall. La historia detrás del error. <https://www.lucidchart.com/blog/es/pros-y-contras-de-la-metodologia-de-cascada>. Último acceso: Octubre 2025.
- [13] Martín Alaimo. Modelo de cascada: Ventajas y desventajas. <https://medium.com/@kleer.la/waterfall-la-historia-dettr%C3%A1s-del-error-39f589a12115>. Último acceso: Octubre 2025.
- [14] Meta. formatoLlama2. <https://www.llama.com/docs/model-cards-and-prompt-formats/meta-llama-2/>. Último acceso: Diciembre 2025.
- [15] Meta. Web oficial de llama. <https://www.llama.com/>. último acceso: Enero 2026.
- [16] Ollama. Ollama. <https://ollama.com/>. último acceso: Julio de 2025.
- [17] Ollama. Ollama. <https://github.com/ollama/ollama>. último acceso: Julio de 2025.
- [18] openwebui. openwebui. <https://docs.openwebui.com/>. Último acceso: Diciembre 2025.
- [19] pytorch. pytorch. <https://pytorch.org/>. Último acceso: Diciembre 2025.
- [20] Unsloth. Unsloth. <https://unsloth.ai/>. último acceso: Septiembre de 2025.
- [21] Younes B Tim Dettmers Artidoro Pagnoni Sylvain Gugger Sourab Mangrulkar. Making LLMs even more accessible with bitsandbytes, 4-bit quantization and QLoRA. <https://huggingface.co/blog/4bit-transformers-bitsandbytes#:~:text=QLoRA%20tuning%20is%20shown%20to,the%20power%20of%20QLoRA%20tuning>. último acceso: Junio de 2025.